Knowledge Graphs from Unstructured Text

Will Ware

Contents

Preface **1**

Part I: The Landscape **3**

Chapter 1: Why do we want to build Knowledge Graphs? **5**
Large Language Models Work Great 5
Until They Don't 5
The Scale of the Problem 6
Retrieval-Augmented Generation or "RAG" 6
Graph RAG 7
Why Bother Building One? 7

Chapter 2: A Brief History of Knowledge Representation **9**
The Idea That Wouldn't Die 9
What the Frog's Eye Tells Us 10
Semantic Networks and the Frame Problem 11
Meaning is Relational 13
The Logic Wars 15
Doug Lenat and Cyc 16
The Semantic Web and the RDF Era 20
The Industrial Knowledge Graph 21
The Extraction Bottleneck 23
The Robot Scientist 24
The Cross-Disciplinary Machine 25
The Gap Is the Book 26

Chapter 3: What Is a Knowledge Graph, Really? **29**

A working definition in four parts 29
Nodes, Edges, and What They Mean 30
The Semantics Matter 30
Provenance and the Epistemics of a Fact 32
Identity: The Hard Problem 33
What a KG Is Good For 33
Build vs. Buy 35
What a KG Is Not Good For 36

Chapter 4: Representation Is Reasoning 39
From fluency to grounded reasoning 39
Experts have internal knowledge graphs 39
Grounded representation as the fix 41
Provenance, auditability, trust 41

Chapter 5: The Extraction Problem 43
Humans are smarter than you think 43
NLP was insufficient to the task 44
Economics of LLMs for extraction 46

Part II: LLMs Change the Equation 49

Chapter 6: LLMs Make This Practical Now 51
The Economics Argument First 52
What LLMs Actually Are, For This Purpose 53
The Prompt as Schema Binding 55
Handling What Classical Systems Couldn't 56
The Remaining Limitations, Honestly 58
Why This Moment 59

Chapter 7: The Free KG Cases 63
When You Don't Need Extraction 63
Lab Instruments and Measured Data 64
Generated and Synthetic Graphs 65
Curated Graphs at Scale 65
What These Cases Teach Us 66
Hybrid Approaches 67

Chapter 8: Designing Your Schema 69
Schema Design as Intellectual Work 69

Entities: What Gets to Be a Thing 70
Relationships: Meaning and Direction 71
Hierarchy and Inheritance 71
Provenance as a First-Class Schema Concern 72
Designing for Extraction 72
Designing for Evolution 73

Part III: Building It **75**

Chapter 9: Diagnostic Tools **77**

Chapter 10: Design Priorities **83**
Provenance 83
Confidence as a Signal, Not a Guarantee 85
Multi-Source Relationships 86
Provenance at Query Time 86

Chapter 11: The Identity Server **89**
Identity Is Load-Bearing 89
The Scale of the Problem 90
Canonical IDs as Primary Keys 91
Authority Lookup 91
The Lookup Chain 92
Provisional Entities and Promotion 93
Provenance-Derived Entities and the Citation Graph 94
Responsibilities 96
Abstract Interface 96
Domain-Pluggable Behaviour 100
Survivor Selection 100
Entity Status 101
Merge × Promotion Status Rules 101
Idempotency Contract 101
Recommended Implementation: Postgres-Backed 102
Locking Strategy 102
Authority Lookup Caching in `resolve` 103
Synonym Detection via pgvector 103
Schema Notes 104
Multi-Replica Deployment 104

Chapter 12: The Ingestion Pipeline **105**

Why Multiple Passes at All 105
Parsing: Getting to Text 106
Extraction: The LLM Pass 107
Vocabulary: Building a Shared Terminology 108
Deduplication 108
Assembly 109
Progress Tracking and Resumability 110
Design Principles 110
Stages 111
1. Fetch 111
2. Extract 111
3. Ingest 111
Work Queue 112
Paper Artifact Files 113
Parallelism 114
Batch Ingestion 114
MCP Tool 115
Extraction Output Format 116
Shared Pipeline Code 118

Part IV: What It Makes Possible 121

Chapter 13: What Your Graph Can Do 123
The Server Is Not the Point 123
Direct Querying 124
Graph Visualization 124
Grounding LLM Inference 125
MCP as the Integration Point 126
BFS Queries 126
Hypothesis Generation 134
Should Hypothesis Generation Be Baked In? 135
Returning to the Dream 136

Chapter 14: The Augmented Researcher 139
What Machines Would See That We Can't 139
The Combinatorial Argument 140
Linguistic and Geographic Blind Spots 141
The Robot Scientist, Revisited 142

Chapter 15: Who Benefits, Who Decides **145**
Democratization and Its Limits 145
Compressed Discovery Timelines 146
The Rare Disease Problem 146
Credit, Priority, and Provenance 147
Who Owns the Graph 147

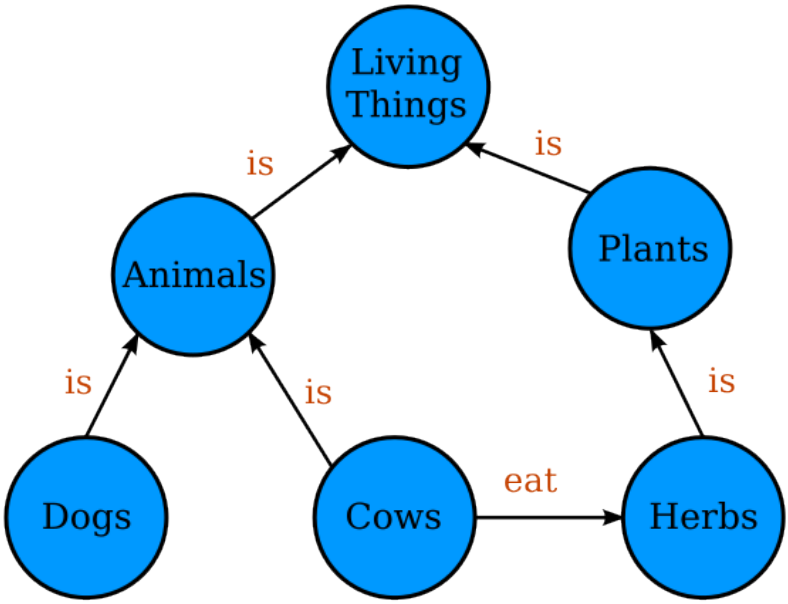
Chapter 16: The Inference You Didn't Intend **149**
The Architecture of Expertise 149
Returning to HAL 149
Dual Use at Graph Scale 150
Bias at Scale 151
The Epistemic Responsibility of the Builder 151

Chapter 17: What's Next **153**
Open Problems 153
Where the Field Is Going 154
An Invitation 155

References **159**

Preface

This is a book written in the age of Large Language Models, but the central thesis of this book is about machine reasoning in general, now and in the future. Knowledge graphs predate LLMs and will outlast them, because they capture something essential to how humans understand and reason about complex fields in an explicit, structured form that can be shared and curated. A machine cannot reason reliably about such fields without knowledge encoded in some form of graph. The software projects described here are demonstrations of this thesis, not the subject of it. LLMs are enablers for the creation of knowledge graphs, which were much discussed in the past but only practical at scale now.



This is a picture of a simple knowledge graph. Bubbles and arrows (we can call them "nodes" and "edges" or "vertices" and "edges") both have labels representing metadata. Graph theory is a branch of mathematics that includes non-trivial theorems and results.

The gist of this book (beyond providing a lot of how-to information) is that a knowledge graph in some form is

- a necessity, not just a convenience, for reliable machine reasoning
- an explicit representation of how human experts understand difficult topics

Part I: The Landscape

Chapter 1: Why do we want to build Knowledge Graphs?

Large Language Models Work Great

We ask them questions about the capitals of countries, or about a chemical formula, or how long to bake something in the oven, and usually we get an answer that is articulate, confident, intelligent-sounding, and correct. We can go to Wikipedia or Google and confirm that, yes, that is the right answer. It's a feel-good moment. The fluency is real. The models have internalized an enormous amount of surface pattern from human text, and for a large class of questions that pattern is enough.

Until They Don't

Correctness was never a primary design priority for LLMs. They are neural networks trained on large corpora. They try to predict the next bit of text, following statistical patterns derived from the training set. As long as our questions stay well within the training set, we can expect answers that are correct most of the time.

When we stray outside the training set contents, the LLM has no mechanism or structure to gauge correctness and no way to correct an answer. We get answers that are articulate, confident, intelligent-sounding, and wrong. My brother told an LLM:

> I live near a carwash and the weather is warm and sunny.

> I want to get my car washed. Should I walk or drive there?

and of course he was told that on a nice day like this, he could use the exercise, so he should walk to the carwash.

The LLM only "knows" what's in the training set, only the statistical patterns of the text it was trained on. It doesn't know that he'll need his car in order to get it washed. The dangerous part is that the wrong answer was delivered with the same tone and confidence as a right one would have been. There is no internal signal that says "I'm extrapolating here" or "I'm not sure." The machine has no way to distinguish a retrieval from memory from a plausible guess. This is **hallucination** – the model producing confident, fluent, false output because it is doing what it was designed to do (generate statistically plausible text) in a situation where the right answer is not well represented in its training. Hallucination is not a bug to be patched; it is a predictable consequence of how LLMs work.

The Scale of the Problem

For casual use, hallucination might be acceptable. For anything that matters – medical advice, legal research, scientific synthesis, technical decisions – we need more than fluency. We need answers that are grounded in something checkable, that can be traced to a source, that can be updated when the world changes, and that reflect the structure of the domain rather than the statistics of the training corpus. That is a different kind of system.

Retrieval-Augmented Generation or "RAG"

We can artificially extend the scope of the training set by adding content to the prompt for the parts the LLM is likely to get wrong. My brother might have created a prompt describing car wash operations and mentioning that the car must be physically present for the operations to work. With the prompt extended in this way, eventually the LLM would stop making that kind of mistake. That would have been a laborious manual process of tinkering and re-wording, and seeing what worked best. This approach would not scale to large bodies of knowledge.

In practice, RAG usually means retrieving relevant passages from a document store and stuffing them into the prompt. That helps: the model can reason over the retrieved text instead of relying solely on training. But retrieved passages are still just text. The model has to parse them, resolve references, and combine information across snippets on the fly. There is no explicit representation of *what* entities are in play or *how* they are related. The structure of the domain stays implicit in the prose, and the model is left to infer it every time. For narrow, one-off questions that can be answered from a few paragraphs, this often works. For complex reasoning that depends on many entities and relationships, or for questions you didn't know to ask in advance, passage retrieval hits its limits.

Graph RAG

The LLM is given a knowledge graph to consult. Instead of raw passages, it gets entities and typed relationships: this drug *treats* this condition, this gene *encodes* this protein, this study *reports* this finding. The graph answers "what is connected to what" and "what kind of connection is it" in a form the model can traverse and cite. The entities and the links between them provide facts, context, names, dates, and meaningful connections. You knew you were asking an egg question for your omelette but you didn't realize in advance that you might also want to know how to tell if an egg has gone bad; the graph can surface that connection because the structure is explicit.

A knowledge graph built from your domain gives the model something to reason *from* rather than something to paraphrase. Claims can be traced to sources. Gaps and conflicts in the graph are visible. When the underlying evidence changes, you update the graph instead of retraining the model. The graph is a shared, inspectable representation of what the system is allowed to "know" in that domain.

Why Bother Building One?

Knowledge graphs provide a unique return on investment. They are simple data structures, easy to understand, not too difficult to build with the tools we have now, and easy for an LLM to query. They

reflect the shape of human knowledge with surprising accuracy when the extraction is done well. The rest of this book is about when you want one, how to design it, and how to build it from the unstructured text where most of that knowledge still lives.

Chapter 2: A Brief History of Knowledge Representation

There are three ideas running through this history.

1. Reasoning benefits from (and may be impossible without) preprocessing of input data to get it into a tractable structure that a computer (or frog brain) can understand.
2. Meaning does not inhere in isolated labels or symbols, it emerges from the linking of many labels or symbols to one another in specific ways to form a network.
3. Building a linked network out of text that has no such preexisting structure is a hard problem, particularly at scale (thousands or millions of documents).

The Idea That Wouldn't Die

There is a fantasy at the heart of computing that is almost as old as computers themselves: the machine that doesn't just store and retrieve facts but *understands* them. Not a filing cabinet you query with the right syntax. Not a search engine that hands you links and wishes you luck. A machine that knows things the way a person knows things – that can be asked a question in plain language, draw on what it understands about the subject, and tell you something true and useful in return.

This fantasy has motivated some of the most ambitious projects in the history of computer science. It has attracted brilliant people, ab-

sorbed enormous funding, and produced genuine results – and then, repeatedly, stalled. Not because the researchers were wrong about what they were trying to build. Because building it turned out to be much harder than it looked, for reasons that took decades to fully understand.

That history is worth understanding, because the stall always happened in the same place. Not at the reasoning end – humans got surprisingly far at encoding the *logic* of a domain, the rules and relationships and inference patterns that experts use. The wall was always at the other end: getting knowledge *in*. Turning the vast, messy, ambiguous record of what humans know – written in papers and books and case notes and specifications, in the imprecise and context-dependent medium of natural language – into something a machine could actually reason over. That problem defeated every approach until very recently.

This chapter is the story of those attempts. It is not a story of failure. The people who built expert systems in the 1980s, who designed the Semantic Web in the 1990s, who curated Freebase in the 2000s – they were right about what was needed. They were working with the tools available to them. The story is worth telling because in retrospect we can see the same general idea approached from different directions, stalling at the same bottleneck, until the idea could finally get the traction it needed.

What the Frog's Eye Tells Us

In 1959, a paper appeared in the *Proceedings of the IRE* with an unusually vivid title: "What the Frog's Eye Tells the Frog's Brain." (Lettvin et al. 1959) Its authors were Jerome Lettvin, Humberto Maturana, Warren McCulloch, and Walter Pitts, and its central finding was startling enough that it still rewards a careful reading sixty-five years later.

The experiment was straightforward in concept. Lettvin and his colleagues placed electrodes in the optic nerve of a frog and observed which retinal ganglion cells fired in response to which visual stimuli. What they found was that the frog's retina doesn't send the brain a raw image. It sends the brain *processed features* – small dark moving objects (bugs), large dark approaching shapes (preda-

tors), sharp edges, sudden dimming. By the time the signal reaches the frog's brain, the eye has already done substantial interpretation. The retina is not a camera. It's a feature extraction pipeline.

The implications reached beyond frog vision. If perception in even a simple vertebrate is not passive reception of raw data but active, structured, selective extraction of semantically meaningful features, then the prevailing model of how biological intelligence works – sense first, interpret later – was badly wrong. Meaning isn't added to perception after the fact. It's built into the extraction process itself. Intelligence, biological or artificial, doesn't work by accumulating raw data and reasoning over it afterward. It works by extracting structured signals and reasoning over *those*.

What the frog's eye tells us is that the process of importing data must impose structure on that data if subsequent reasoning is to succeed.

Semantic Networks and the Frame Problem

The graph as a mathematical object is ancient. Euler's 1736 solution to the Königsberg bridge problem – can you cross each of the city's seven bridges exactly once? – is usually cited as the birth of graph theory, and the centuries of results that followed established it as a mature branch of mathematics long before anyone thought to use it for knowledge representation. Euclid's *Elements*, two thousand years earlier, had definitions, propositions, and logical dependencies between them that you could draw as a directed acyclic graph – but Euclid wasn't making a claim about knowledge representation, he was doing geometry. The data structure, in some form, has always been available. The question was whether anyone would recognize it as the right structure for something other than bridges and triangles.

The answer, in the context of AI, arrived in 1966 with Ross Quillian's doctoral dissertation on semantic memory (Quillian 1966). Quillian was trying to represent the meanings of words – not as dictionary definitions but as structured networks of relationships. The word "plant," in Quillian's model, wasn't defined by a string of text; it was a node connected by labeled links to other nodes representing its properties, its categories, its relationships to other concepts. This was recognizably a knowledge graph in embryonic form, and it established the semantic network as a tool for AI research.

Marvin Minsky's contribution, in his 1974 paper "A Framework for Representing Knowledge" (Minsky 1974) was different in kind. He wasn't just proposing a data structure – he was making a cognitive claim. The claim was that when humans encounter a situation, they don't process it from scratch. They retrieve a pre-existing structure – a frame – that represents a stereotypical version of that kind of situation, with slots for the details that vary. A "restaurant" frame has slots for host, menu, food, check, tip. When you walk into a restaurant you don't reason from first principles about what's happening – you instantiate the frame and fill in the slots from what you observe. The frame carries default values for unfilled slots, which is how you know to expect a menu before one appears.

What made Minsky's paper feel vague to readers who wanted a specification – and it did feel vague, the reaction was common and not a failure of reading – was that he deliberately left open how frames were organized, how the right frame gets retrieved, how conflicts between frames get resolved, how new frames get created. These are hard problems and he largely left them open. He was sketching a research agenda, not specifying a system. In retrospect this was the right choice: premature formalization of a half-understood idea produces a wrong formalism that is harder to correct than a productive sketch. Minsky was doing what good theorists do at the frontier – pointing confidently at something real without pretending to have fully characterized it.

The frame problem – related to but distinct from Minsky's frames – is the AI challenge of representing what *doesn't* change when something happens. If you move a block from one table to another, the block's color doesn't change. This seems obvious; it's extraordinarily difficult to formalize in a way that scales. Every action in a world has a vast number of things it doesn't affect, and a reasoning system that has to explicitly represent all of them is overwhelmed before it begins. The frame problem was one of the first demonstrations that human common sense, which handles this effortlessly, is hiding enormous computational complexity.

What frames and semantic networks got right – what makes them the direct ancestors of the knowledge graph – is the claim that the relationships *are* the knowledge. Not an index of facts with relationships as an afterthought, but a structure in which meaning is

constituted by connection. A node in isolation is just a label. A node embedded in a graph of typed relationships to other nodes is a concept, with implications, with context, with a place in a web of meaning. This is not just a useful engineering observation. It is, as Minsky was arguing, a description of how intelligence works – how a mind, human or artificial, can know what it knows in a way that supports inference rather than mere retrieval.

The knowledge graph as built today is in many ways the realization of what Minsky was gesturing at: a rigorous, computable, queryable structure where entities have typed relationships, where context shapes interpretation, where the graph itself carries meaning rather than merely pointing at it. The difference is that Minsky was working at the level of cognitive theory, and we are building infrastructure. The informality that frustrated impatient readers in 1974 has become, fifty years later, a schema definition language and a pipeline.

Meaning is Relational

Douglas Hofstadter's argument in *Gödel, Escher, Bach* (Hofstadter 1979) is directly relevant here, and the connection is tighter than it might first appear.

Hofstadter's central concern is how meaning arises in formal systems – how symbols, which are just patterns, come to *refer* to things in the world. His answer, developed through the interweaving of Gödel's incompleteness theorems, Escher's self-referential drawings, and Bach's fugues, is that meaning isn't a property of individual symbols but of *symbol systems* – of the relationships and transformations between symbols. A symbol means something because of its position in a web of other symbols and the rules that connect them. Isolate the symbol from that web and the meaning evaporates.

This is precisely the claim that frames and semantic networks were making, and that knowledge graphs embody. A node in a knowledge graph isn't meaningful in isolation – "BRCA1" as a string of characters means nothing. It means something because of its typed relationships to other nodes: it *encodes* a protein, it *is associated with* breast cancer risk, it *interacts with* other genes, it *has synonym* "breast cancer type 1 susceptibility protein." The meaning is in the

web, not in the label.

Hofstadter goes further in a way that connects to Chapter 4's argument. He distinguishes between what he calls *active symbols* and passive ones. A passive symbol is just a token that gets manipulated by external rules – the symbols in a formal logical system, for instance. An active symbol is one that participates in its own interpretation, that has something like internal structure that fires in response to the right context. His argument is that genuine intelligence requires active symbols – that what brains do, and what any sufficiently sophisticated reasoning system must do, is maintain a network of active symbols that mutually constrain and activate each other.

The knowledge graph, in this framing, is an attempt to make active symbols computationally tractable. Not just a passive lookup table but a structure where the typed relationships do real work – where querying "what treats this disease" is not a string match but a traversal of a semantic network that carries the meaning of "treats" as a typed predicate with defined subject and object constraints. The graph doesn't just store the claim; it participates in the reasoning.

There's a passage in GEB where Hofstadter discusses the difference between the *map* and the *territory* – between a representation and the thing it represents – and argues that sufficiently rich representations develop a kind of isomorphism with their subject that amounts to genuine understanding. The representation isn't just *about* the territory; it captures enough of the territory's structure that reasoning over the representation produces true results about the territory. This is, almost word for word, the argument your Chapter 4 is making about knowledge graphs and grounded inference.

Hofstadter gave us the theoretical vocabulary for why symbol systems need to be relational rather than atomic, why meaning is constituted by connection rather than inherent in labels, and why a sufficiently structured representation can support genuine inference rather than mere retrieval. Minsky gave us the cognitive architecture – frames as the natural unit of structured knowledge. The knowledge graph is what you get when you take both of those arguments seriously and ask: what does this look like as buildable infrastructure?

The Logic Wars

The expert systems of the 1970s and 80s were not a wrong turn. They were a genuine insight, implemented with the tools available, and in the right conditions they worked remarkably well.

The insight was this: if you could get a domain expert to articulate the rules they used to reason – the *if-this-then-that* chains that an experienced diagnostician or engineer carried in their head – you could encode those rules in a formal system and the machine could apply them. MYCIN, the system built at Stanford in the early 1970s to diagnose bacterial infections and recommend antibiotics, could outperform medical residents on its target task. XCON, deployed by Digital Equipment Corporation to configure VAX computer systems, was saving the company tens of millions of dollars a year by the early 1980s. These weren't demos. They were production systems doing real work.

The problem was at the edges of the recorded knowledge domain. A domain expert can articulate the rules they consciously apply, but expertise isn't just conscious rules – it's also the vast background of common sense and contextual judgment that experts exercise without noticing they're doing it. Ask a doctor what rules she uses to diagnose a bacterial infection and she can tell you. Ask her what rules she uses to know that a patient who says "I feel fine" while running a 104-degree fever is not, in fact, fine – and she'll struggle, because that knowledge isn't stored as rules. It's pattern recognition built from years of experience, running below the level of explicit articulation.

Expert systems couldn't capture that. They could only encode what the expert could say out loud. Everything else – the background assumptions, the common sense, the contextual adjustments – had to be anticipated and written down by a knowledge engineer. And it turned out that the space of things you had to anticipate was essentially unbounded. Add a new drug to MYCIN's domain and suddenly you needed new rules, and new rules to handle the interactions with old rules, and new rules to handle the exceptions to those interactions. The system that worked beautifully on the original narrow problem became brittle the moment the world asked it a question slightly outside its prepared territory.

Prolog and the description logics that followed were attempts to put the reasoning on a more rigorous mathematical footing. If the problem with expert systems was that the rules were ad hoc and hard to maintain, maybe formal logic would help – a cleaner representation that could be reasoned over systematically, whose consequences could be derived rather than enumerated. The logic was sound. The problem was the same: you still had to get the knowledge in. Formal logic made the representation more principled, but it didn't make knowledge acquisition any less expensive or any less brittle when the domain proved more complex than the ontology anticipated.

By the early 1990s, the expert system boom had become the AI winter. Funding dried up, companies that had bet heavily on the technology quietly wrote off their investments, and the field moved on. What it left behind was a lesson that wouldn't fully land for another thirty years: the bottleneck was never the reasoning. It was always importing the knowledge at scale.

Doug Lenat and Cyc

Douglas Lenat arrived at the Cyc project by a route that is worth tracing, because the route explains the ambition.

His earlier systems – AM and Eurisko – were investigations into machine creativity: could a program discover new concepts and heuristics by exploring a space of possibilities, guided by rules about what makes a discovery interesting? AM, built in the mid-1970s, started from the basic concepts of set theory and rediscovered, by its own heuristic search, a surprising fraction of elementary number theory – prime numbers, divisors, the concept of maximally divisible numbers. It wasn't doing mathematics in any deep sense; it was pattern-matching over a space of concept definitions using rules Lenat had carefully encoded. But the output was striking enough to attract serious attention.

Eurisko extended the idea. Where AM's heuristics were fixed, Eurisko's could modify themselves – it could discover new rules for what made exploration productive and apply them. This led to the incident that made Lenat briefly famous outside academic circles. Eurisko was entered in the Traveller TCS tournament, a national naval fleet design competition played on paper. In 1981, it won, having

designed a fleet that exploited an undervalued tradeoff in the game's rules – a large number of small, cheap, expendable ships rather than the expensive dreadnoughts human players favored. The tournament organizers changed the rules. In 1982, Eurisko was entered again and won again, having found a different undervalued tradeoff in the new rules. The organizers made clear that future entries by the system would not be welcome.

The lesson Lenat drew from Eurisko was not primarily about heuristics. It was about the limits of heuristics without knowledge. Eurisko could explore effectively within a closed, well-defined space – a game with explicit rules and a clear scoring function. It had no way to transfer that ability to any domain that wasn't already fully formalized. The world, obviously, is not a game with explicit rules and a clear scoring function. What was missing was common sense: the vast background of understanding that humans apply effortlessly to almost everything they do, and that is nowhere written down in any form a machine can use.

In 1984, Lenat joined MCC – the Microelectronics and Computer Technology Corporation, a research consortium set up in Austin, Texas, as the American answer to Japanese fifth-generation computing ambitions – and began what would become the longest-running project in the history of artificial intelligence. The goal of Cyc, as he stated it, was to encode enough common sense knowledge that the system could eventually read text and teach itself. The bootstrapping problem was explicit from the start: you couldn't teach a machine to learn from natural language until you had given it enough background to understand what it was reading. The hand-encoding phase was, in Lenat's framing, an unavoidable investment – a kind of pre-training that had to be done by people before the machine could take over. His initial estimate was that this would require roughly 250,000 rules and about 2,000 person-years of effort.

The knowledge representation language developed for Cyc was CycL, and it is worth understanding what it was trying to do. CycL is a variant of first-order predicate calculus – the formal logical language in which you can state things like "for all X, if X is a person, then X has a mother" – extended with higher-order constructs to handle the cases where first-order logic proves insufficient. A CycL statement looks something like this: `(isa DougLenat Person)`, `(birthDate`

`DougLenat (MonthFn July 1950)), (beliefs DougLenat (ist CycCorpMt (capable Cyc understands NaturalLanguage)))`. The predicates, the constants, and the relationships between them are all explicitly typed, with constraints on what can appear as subject and object of each predicate.

What made CycL genuinely novel, and what made Cyc structurally different from the expert systems of the same era, was the notion of microtheories. The problem that sinks naive attempts to encode common sense as a flat knowledge base is that the world is context-dependent in ways that produce apparent contradictions. Water is a liquid. Water is also a solid – if you're talking about the Arctic. An organization can have a president who is also a person who runs an organization, but "the president of the United States" and "the president of the chess club" require different inferences. In a flat knowledge base, these contextual dependencies are difficult to represent cleanly and even more difficult to reason over without contradiction.

Microtheories were Cyc's solution. A microtheory is a named context, a compartmentalized set of assertions that hold within a particular domain, time period, or frame of reference. `CycPhysicsMt` contains assertions about physical laws. `FictionMt` contains assertions about fictional entities. `NaivePhysicsMt` contains the commonsense approximations that humans apply in everyday reasoning, which are not the same as what a physicist would assert and do not need to be. Assertions made in one microtheory do not automatically propagate to others, and the inheritance relationships between microtheories are themselves explicit. This allowed Cyc to handle what philosophers call the frame problem – the problem of representing what is *not* changed by an action, and what is *not* true in a context – in a way that expert systems with flat rule bases could not.

The manual encoding effort was enormous, tedious, and philosophically treacherous. Knowledge engineers at MCC and later at Cycorp (the spin-off company formed in 1994) spent years encoding assertions that any child knows: that physical objects exist in only one place at a time, that people can't read books they haven't seen, that most people have two parents, that fire is hot, that rooms have floors. Each assertion was in some sense trivial. The challenge was that there are an essentially unbounded number of them, they interact

in ways that are difficult to anticipate, and natural language is full of assumptions built on other assumptions that must themselves be encoded before the first one makes sense. By the time Cyc was several decades old, it contained hundreds of thousands of concepts and millions of assertions. The hand-encoding phase, the phase that was supposed to precede the self-learning phase, never ended.

This is the honest verdict on Cyc, and it should be delivered without condescension, because the project understood the problem better than almost anyone. Lenat was right that common sense was the bottleneck. He was right that background knowledge had to be formalized before a machine could reason reliably. He was right that the formalization required an explicit, queryable, logically consistent representation – not a pile of text, not a set of statistical weights, but something structured enough to support inference. He was wrong, or at least optimistic, about how long the bootstrapping phase would take and whether it would ever terminate. The reading goal – the system teaching itself from text – was never reached, not because the goal was misconceived but because the amount of background knowledge required to understand even simple sentences turned out to be larger, and more entangled with itself, than anyone had anticipated. Cyc could read text in the technical sense of processing it; it could not understand text in the sense of reliably extracting the intended meaning without extensive hand-guided support.

What Cyc demonstrated, inadvertently, was the same lesson that every other approach of the era demonstrated from a different angle. The knowledge had to be encoded by people. The encoding was labor-intensive, brittle at the edges, and never quite complete enough to do what you needed it to do. The extraction bottleneck did not yield to formalization; it yielded only to scale, and the kind of scale required was not human-hours-in-Austin but the ingestion of essentially all written human output and the statistical extraction of its implicit structure. That is what Lenat was pointing at when he described the self-learning goal. He was four decades early and the mechanism he anticipated – hand-coded rules as the initial substrate – was not the one that eventually worked. But the destination he identified was real.

Lenat died in 2023. Cycorp continues. OpenCyc, a reduced version of the knowledge base, was released as open source in the mid-2000s

and influenced a generation of researchers in ontology and knowledge representation. The CycL language, the microtheory architecture, and the careful attention to context-dependence and frame problems were genuine intellectual contributions that the field built on even when it moved away from the specific Cyc approach. The project failed to reach its most ambitious goal. So did every other project of the same era. That failure, traced from the right angle, is the argument this book is making.

The Semantic Web and the RDF Era

Tim Berners-Lee invented the World Wide Web and then, almost immediately, started worrying that he'd built the wrong thing.

The web he'd built was for humans. Pages of text and images, linked to other pages, navigable by people who could read and follow links and make inferences about what they were looking at. Machines could fetch the pages, but they couldn't understand them – couldn't know that a page about a restaurant contained a phone number, or that a page about a drug contained a dosage, or that two pages from different sources were talking about the same person. The web was a vast store of human knowledge wrapped in a format that machines could retrieve but not comprehend.

His vision for fixing this, described in a 2001 *Scientific American* article (Berners-Lee, Hendler, and Lassila 2001), was the Semantic Web. The idea was to augment the existing web with structured, machine-readable data, encoded in a standard format called RDF (Resource Description Framework), so that software agents could traverse the web and actually *understand* what they were reading. Not just fetch a page about a drug, but know that the page described a drug, that the drug had a name and a manufacturer and a set of indications, and that those indications were linked to diseases described on other pages. The web of documents would become a web of data.

The vision was sound. The problem was asking people to do a lot of work for benefits that mostly accrued to others.

Encoding your content as structured RDF was significantly more effort than writing HTML. The tools were complex, the standards were baroque, and the payoff for any individual publisher was unclear –

the value of structured data emerges when many sources use compatible schemas, and in 2001 nobody was. The classic adoption problem: the network is worth joining only when enough other people have joined, but nobody wants to be first. Only a few large institutions – libraries, government data publishers, some academic databases – embraced the vision seriously.

What emerged from the wreckage was Linked Data, a more pragmatic interpretation of the same basic idea. Instead of demanding that everyone adopt a comprehensive semantic framework, Linked Data asked only that you use URIs to identify things, link to related URIs from other datasets, and publish whatever structured data you had in whatever simple format you could manage. It was a retreat from the grand vision, but it was a retreat to something that actually worked. DBpedia – a structured version of Wikipedia extracted by the community – became a hub that linked to dozens of other datasets. Wikidata grew into something genuinely useful. The government open data movement produced real structured datasets. A modest version of the dream was alive.

The community sold Linked Data coffee mugs to encourage adoption. It was that kind of movement, idealistic, slightly desperate, held together by enthusiasm and standards documents. The web did not restructure itself accordingly.

But the fundamental problem remained. Linked Data worked well for knowledge that someone had already structured. The vast majority of what the web contained – the papers, the articles, the reports, the case studies, all the text – remained opaque. You could link your structured data about a drug to Wikipedia's article about that drug, but you couldn't automatically extract the structured data from the article in the first place. The extraction bottleneck was still there, just slightly better acknowledged.

The Industrial Knowledge Graph

In May 2012, Google announced what it called its Knowledge Graph with a phrase that became a kind of slogan for the field: the goal was to understand "things, not strings." Instead of matching keywords to documents, Google wanted its search engine to understand that "Leonardo da Vinci" was an entity – a person, with a birthdate, a

nationality, a set of works – and that queries about him were queries about that entity, not just about a sequence of characters. The knowledge panel that appears to the right of search results when you search for a person, place, or organization is the visible surface of this system.

The Knowledge Graph didn't emerge from nowhere. Its initial corpus was built primarily from Freebase, a structured knowledge base that a company called Metaweb had been developing since 2007 and that Google acquired in 2010. Freebase was itself a descendant of the collaborative, community-edited spirit of Wikipedia, but where Wikipedia stored knowledge as prose, Freebase stored it as structured facts: typed entities connected by typed relationships, contributed and edited by users. By the time Google folded it into the Knowledge Graph, Freebase contained roughly 44 million topics and 2.4 billion facts. Google supplemented this with Wikipedia, the CIA World Factbook, and other structured sources. The graph grew rapidly – tripling in size within seven months of launch, reaching 570 million entities and 18 billion facts by the end of 2012, and eventually growing to hundreds of billions of facts on billions of entities.

What Google demonstrated at scale was that a knowledge graph was extraordinarily useful for a specific class of questions: factual queries about well-known entities. Who directed this film? When was this person born? What is the capital of this country? For these queries – where the answer is a fact about a named entity that exists in the structured corpus – the knowledge panel delivers an answer directly, without the user needing to click through to a source. This was genuinely transformative for search, and it established the knowledge graph as production infrastructure rather than academic exercise.

But it also revealed, somewhat quietly, what the approach could *not* do. The Google Knowledge Graph was built almost entirely from structured sources – curated databases, collaborative encyclopedias, government factbooks. It was very good at facts that someone had already explicitly encoded. It had almost no ability to extract knowledge from the unstructured text that makes up the vast majority of what humans have written. Google's own internal research project, Knowledge Vault, attempted to address this – automatically harvesting facts from across the web – but it remained a research effort rather than a production system, and the problem of reliably

extracting structured knowledge from free text remained essentially unsolved.

Freebase itself was shut down in 2016, its data migrated to Wikidata, the open collaborative knowledge base that now serves as one of the primary structured sources for the Knowledge Graph. The arc is instructive: even Google, with essentially unlimited engineering resources, found it easier to rely on human-curated structured data than to solve the extraction problem at the quality level required for production use. The extraction bottleneck wasn't a failure of ambition. It was a genuine hard problem that the tools of the time couldn't crack.

The Extraction Bottleneck

Every era of knowledge representation described in this chapter ran into the same wall, approached from a different direction. The semantic networks of the 1960s and 70s had to be hand-built by knowledge engineers – a process so labor-intensive that the resulting systems covered narrow domains at best. The expert systems of the 80s required domain experts to spend months encoding their knowledge as rules, and still couldn't capture the tacit understanding that experts applied effortlessly. The Semantic Web vision of the 90s and 2000s imagined a world where structured data would be published everywhere – but the incentive for any individual publisher to do the hard work of structuring their content for others' benefit was never quite strong enough. And Google's Knowledge Graph, for all its scale, was built from the structured sources that already existed: databases, encyclopedias, factbooks. It was excellent at encoding what had already been made explicit. It had no way to reach the knowledge that hadn't.

The uncomfortable arithmetic is this: the vast majority of what humanity knows is written down in natural language – in papers, books, reports, articles, case studies, clinical notes, legal opinions, engineering specifications, and a million other forms of text. This is where the knowledge lives. Not in databases. Not in ontologies. In sentences. And sentences, it turns out, are extraordinarily difficult to machine-read at the level required to extract structured, typed, confident knowledge from them.

We'll later return to this sentence taken from the cancer literature. It illustrates some of the linguistic twists and turns that humans handle effortlessly.

> Patients treated with the combination showed significantly reduced tumor burden compared to controls, though the effect was attenuated in those with prior platinum exposure.

The knowledge extraction bottleneck persisted as the tools slowly improved, but the fundamental economics of the problem remained the same. The knowledge was in the text. Getting it out reliably, at scale, across domains, remained out of reach.

The Robot Scientist

In 2009, a paper appeared in *Science* with a deceptively modest title: "The Automation of Science." (King et al. 2009) Its author, Ross King, then at Aberystwyth University, described a system called Adam that had done something no machine had done before: it had conducted original scientific research autonomously, from hypothesis to experiment to conclusion, without a human in the loop.

Adam's domain was the functional genomics of baker's yeast, *Saccharomyces cerevisiae*. Yeast is a well-studied organism, but in 2009 the functions of a significant fraction of its genes were still unknown. Adam was given access to a knowledge graph of yeast biology – the known metabolic pathways, the existing gene-function assignments, the databases of protein interactions – and tasked with identifying genes whose functions could be inferred and tested.

What Adam did, step by step, was this: it reasoned over the knowledge graph to identify gaps – genes that, given what was known about the pathway, *ought* to encode a particular enzyme but for which no such function had been assigned. It formulated these gaps as hypotheses. It designed experiments to test the hypotheses, in the form of instructions to a robotic laboratory system. It ran the experiments. It observed the results. It updated its beliefs about the knowledge graph accordingly. And then it began again.

This was not assisted science. It was not accelerated science in the sense of a human scientist working faster with better tools. The

scientific method – observe, hypothesize, experiment, update – was running autonomously. Adam identified and confirmed the functions of several previously uncharacterized yeast genes. The discoveries were real. They were published. They held up.

King went on to build Eve, a successor system focused on drug discovery, which identified a promising compound for treating malaria by screening molecules against parasite targets in an automated laboratory. Adam and Eve were not the most capable systems ever built for their respective tasks – human scientists with enough time and resources could do what they did. What they demonstrated was that the *process* of science could be formalized and automated, given a rich enough knowledge representation to reason over.

The question that Adam poses – and that this book is, in part, an attempt to answer – is what happens when that knowledge representation is no longer limited to a single well-curated domain, and no longer requires a team of knowledge engineers to build by hand.

The Cross-Disciplinary Machine

Adam's knowledge graph was narrow by design. It covered yeast biology with enough depth to support genuine inference, and it covered almost nothing else. That narrowness was what made the system tractable in 2009 – and it was also the ceiling on what Adam could discover. The hypotheses Adam could form were hypotheses *within* yeast biology. It could notice that a metabolic pathway had a gap and infer what might fill it. It could not notice that a compound studied in a completely different context – say, a natural product investigated for antimicrobial properties in a separate literature – had structural properties that might make it relevant to the yeast pathway it was reasoning about. That connection would require a graph that spanned both domains and knew that the relevant entities in each were related.

The reason cross-disciplinary reasoning is hard isn't that the connections don't exist. They do, and the history of science is full of examples: Fleming noticing that a mold was killing his bacterial cultures, Krebs recognizing the cyclic nature of metabolic reactions by drawing on observations scattered across the biochemistry literature, the structural biologists whose work on crystallography turned out

to be essential to understanding DNA. These weren't lucky accidents. They were pattern matches made by unusually well-read people who happened to have enough breadth to see a connection that specialists couldn't.

A knowledge graph spanning multiple domains can, in principle, make those pattern matches systematic rather than serendipitous. But only if it can recognize that the entities are the same across domains – that the gene discussed in a genetics paper and the gene discussed in an oncology paper are the same node, not two separate strings that happen to look similar. This is the canonical identity problem again, now with stakes beyond bookkeeping: without it, the multi-domain graph isn't a connected network of knowledge, it's a collection of islands. Canonical identity, anchored to established ontological authorities, is what builds the bridges.

This is already happening, in a limited way, at the level of encyclopedic knowledge. Wikidata links entities across domains – a drug, a disease, a researcher, an institution – with stable identifiers that let structured data from different sources be combined and traversed. DBpedia does something similar for the structured content embedded in Wikipedia. These systems are genuinely useful for factual queries that span domains. What they cannot do is reach the research frontier – the findings that are in papers but not yet in encyclopedias, the preliminary results that haven't been replicated enough to earn a Wikidata entry, the connections that haven't been noticed yet because no one has read both papers.

The Gap Is the Book

Adam existed. The cross-disciplinary machine is not a fantasy – it's an engineering problem with a visible shape. The destination is real.

What stands between here and there is the same wall that stopped every previous attempt: getting knowledge in. Adam worked because its domain was narrow enough, and structured enough, that the knowledge could be loaded by hand. Extending that to the full body of published science – millions of papers, across every discipline, in every language, updated continuously as new results arrive – requires something Adam didn't have: a way to read the literature and extract structured, typed, provenance-tracked knowledge from

it automatically, reliably, and at scale.

That is the missing piece. It is also, as of a few years ago, newly within reach. The rest of this book is about how to build it.

Chapter 3: What Is a Knowledge Graph, Really?

A working definition in four parts

These four properties together distinguish a knowledge graph from a labeled graph, a property graph, a document store, or a well-structured relational database. Any of the four can be relaxed for pragmatic reasons – and sometimes they should be – but relaxing them has costs that are worth understanding before you do it.

Nodes represent entities. Discrete, identifiable things: people, places, substances, concepts, events. Not properties, not values, not free-floating text. Things that can be named, referenced, and recognized across sources – and that's the critical word: *recognized.* When two papers mention the same gene, the same drug, the same disease, a knowledge graph can know they're talking about the same thing. That's what makes cross-source reasoning possible at all. Without it, you're not combining knowledge from multiple sources – you're just accumulating text. Entities can be identified by their place in familiar, accepted authoritative ontologies.

Edges represent typed relationships. Not generic connections but semantically defined predicates with a direction and a meaning. An edge labeled "inhibits" between a drug and an enzyme is a different kind of claim from one labeled "co-occurs with," and the distinction is not cosmetic.

Every entity has a canonical identity. A stable identifier that

persists across documents, authors, and time. The same gene is the same node whether a paper calls it by its official symbol, a common alias, or a misspelling. Resolving that multiplicity to a single identity is not bookkeeping; it's what makes the graph useful.

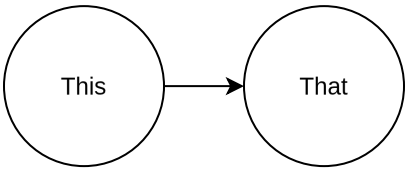
Every relationship carries provenance. A traceable record of where the claim came from, by what method it was established, and with what confidence. A relationship without provenance is an assertion of unknown quality. A relationship with provenance is evidence.

Nodes, Edges, and What They Mean

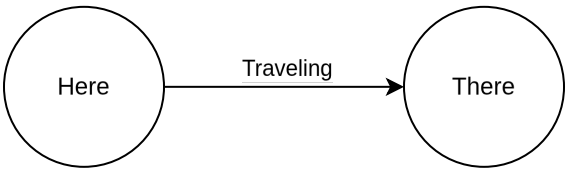
Entities, relationships, and the critical point that a relationship isn't just a link. What distinguishes a KG from a property graph, a relational database, or a document store.

The Semantics Matter

An edge without a type is nearly useless. Knowing that a drug and a disease are *connected* tells you almost nothing; knowing that the drug *treats* the disease, or *causes* it, or *is contraindicated in* patients with it, tells you something actionable. The meaning of the edge is the knowledge. The edge itself is just the plumbing.



An edge with no label or metadata tells us very little, just that two things are connected in some way.



Even a very simple label on an edge gives us a more engaging narrative.

Software engineers will recognize this intuition from type systems. An untyped variable that could hold anything is harder to reason about than one whose type tells you what operations are valid on it and what guarantees it carries. The same principle applies to edges in a knowledge graph: a typed relationship isn't just a label, it's a contract. It tells you what the subject and object are allowed to be, what direction the relationship runs, and what it means to assert it. A graph with well-typed edges is one that can catch its own errors – an "inhibits" edge between two diseases, for instance, is probably a mistake, and a schema that defines valid subject and object types for each predicate will surface that mistake rather than silently incorporate it.

This seems obvious stated plainly, but it has consequences that are easy to underestimate during schema design. The temptation – especially early, when you're trying to get extraction working at all – is to start with a small number of generic relationship types and plan to refine them later. "Associated with" is the classic offender: it's easy to extract, it's never wrong, and it's almost never useful. A graph full of "associated with" edges is a graph that can support retrieval but not reasoning.

The flip side is also real: relationship types that are too narrow make extraction impractical. If your schema requires the model to distinguish between "directly inhibits," "allosterically inhibits," and "competitively inhibits," you've created a precision that the extraction step probably can't reliably deliver, and you'll spend more time correcting misclassifications than you'll gain from the distinction. The right level of semantic granularity is the one where the types are meaningfully distinct, expressible in natural language to an extraction model, and actually supported by what your sources say.

There's a third consideration that doesn't get enough attention: direction. "Drug A treats Disease B" and "Disease B is treated by Drug A" are the same fact, but in a directed graph they're different edges. Getting direction consistent across extraction runs and across sources is a detail that causes real headaches if it's not settled early. Treating direction as part of the type definition – not just a convention but a constraint – keeps the graph coherent as it grows.

Provenance and the Epistemics of a Fact

Every edge in a knowledge graph is a claim. And claims, unlike data, have an epistemological status: they come from somewhere, they were established by some method, they are more or less certain, and they may conflict with other claims made by other sources. Provenance is the machinery that tracks all of this.

At minimum, provenance answers: *where did this relationship come from?* – which source document, which section, which passage. But a well-designed provenance system goes further. It records *how* the relationship was established – was it extracted by an LLM, identified by a named entity recognizer, asserted by a human curator? It records *confidence* – not as a precise probability but as an ordinal signal about how much weight to give the claim. And in domains where it matters, it records the *epistemic type* of the evidence – a randomized controlled trial is a different kind of claim from a case report, which is a different kind of claim from a computational prediction.

This last point deserves emphasis. In medicine, the Evidence and Conclusion Ontology (ECO) exists precisely because "there is evidence for this relationship" is not a single thing – it's a spectrum from "one lab observed this once under unusual conditions" to "this has been replicated across fifty independent studies in multiple populations." A knowledge graph that conflates these is not just imprecise – it's dangerous. A knowledge graph that preserves the distinction gives downstream reasoning systems something to work with.

The practical consequence is that provenance shouldn't be an afterthought bolted onto a graph that was designed without it. It should be a first-class schema concern from the start. In kgraph, this meant treating evidence as its own entity type – not just a property of a relationship, but a node in the graph with its own identity, its own source, and its own relationship to the claim it supports. That design decision turns out to have large downstream implications: it makes provenance queryable, traversable, and aggregable across sources in ways that a simple confidence score attached to an edge cannot be.

A graph without provenance is a collection of claims with no way to evaluate them – no way to ask "how well-supported is this?", no way to debug "why does the graph believe this?", no way to detect when

two sources are in conflict rather than in agreement. Provenance is what separates a knowledge structure from a very large list of sentences that someone decided to believe.

Identity: The Hard Problem

Canonical identity as a first-class concern, not an implementation detail. Identity is hard and consequential, being introduced at the level of abstraction appropriate for a chapter that's defining what a knowledge graph is. It's non-trivial, and treating it as an afterthought is a mistake. A graph where "BRCA1," "breast cancer gene 1," and "BRCA1 protein" are three separate nodes isn't a knowledge graph – it's an index.

But the stakes go deeper than deduplication. Canonical identity doesn't just help you say that two things are the same. It places those things within the body of human knowledge. The identifiers that matter – MeSH or UMLS CUI for medical concepts (UMLS is perhaps the more complete resource, but requires a licensed API key; for development purposes MeSH is free and sufficient), Gene Ontology terms for molecular function, DBPedia URIs for cross-domain entities – come from accepted authoritative ontologies. They are maintained by communities of experts, revised through consensus, and trusted precisely because they represent the accumulated judgment of the field. When you assign a canonical ID to an entity, you are not merely collapsing synonyms. You are connecting that entity to the history of human thought as far as that entity is concerned: its definition, its place in the taxonomy, its relationships to other concepts that the community has already established and agreed upon. A knowledge graph built on canonical IDs is not just a graph of facts – it is a graph that inherits the epistemic authority of the ontologies it anchors to. That inheritance is what makes the graph trustworthy across sources, across time, and across the boundary between human expertise and machine reasoning.

What a KG Is Good For

Querying. A knowledge graph supports structured queries over entities and relationships in ways that free-text search and document

stores cannot. You can ask "what drugs treat this disease?" or "what genes are implicated in this pathway?" and get answers that are aggregated across sources, deduplicated, and typed. The query is over the structure of the domain, not over the surface form of the text. That distinction matters: a search engine returns documents that might contain the answer; a knowledge graph returns the answer, with provenance pointing back to the documents that support it.

Traversal. The graph structure enables path-finding and multi-hop reasoning. You can ask not just "what does A connect to?" but "how is A related to B?" – and the graph can return paths that span multiple edges and intermediate entities. Those paths often reveal connections that no single source states explicitly: the drug and the disease might never appear together in one paper, but the graph can connect them through shared targets, shared pathways, or shared mechanisms. Traversal is what turns a collection of facts into a navigable map of a domain.

Hypothesis generation. Because the graph makes structure explicit, it surfaces patterns that weren't in the original sources. Drug repurposing – finding that a drug approved for one indication might work for another – often starts with noticing that two diseases share a mechanism, a target, or a pathway. A graph that connects drugs, targets, diseases, and mechanisms can suggest those connections. So can a human expert with years of training; the graph can do it systematically, at scale, and in a form that can be checked. The hypotheses still require validation – the graph proposes, it doesn't prove – but it narrows the search space from "everything we might try" to "things that are structurally plausible."

LLM grounding. A large language model reasoning from its training distribution has no way to distinguish what it actually knows from what it has statistically absorbed. Give it a knowledge graph to reason over, and the task changes: the model retrieves relevant subgraphs, synthesizes them, and produces answers that are grounded in explicit, provenance-tracked claims. The model's role shifts from "remember and generate" to "retrieve and synthesize." That shift reduces hallucination, makes the reasoning traceable, and gives downstream users something to audit. We return to this in Chapter 4; for now, the point is that grounding is not a minor application of knowledge graphs but one of their primary use cases in the current

AI landscape.

Build vs. Buy

The landscape of existing knowledge graphs is richer than it used to be. Wikidata offers broad coverage across many domains, with community curation and a flexible schema. Domain-specific graphs like SPOKE (drug-disease-gene) and ROBOKOP (pharmacogenomics) provide biomedical structure that general-purpose graphs don't. Commercial offerings from publishers, vendors, and platform providers add proprietary value and integration. The question is not whether knowledge graphs exist – they do – but whether one of them fits your problem.

When existing graphs are sufficient. If your domain is well-covered, your schema aligns with what the graph provides, and you don't need provenance that traces back to your own corpus, an existing graph may be the right choice. You get coverage, maintenance by someone else, and a shorter path to value. The tradeoff is that you inherit someone else's design decisions: their entity types, their relationship vocabulary, their choices about what to include and what to leave out. If those align with your use case, that's fine. If they don't, you'll spend time working around them.

When you need to build. Several situations push you toward building your own. *Novel domains* – legal documents, niche scientific subfields, internal corporate knowledge – often have no suitable public graph. *Proprietary corpora* matter when the knowledge you care about lives in documents you control: internal reports, unpublished studies, patient records, contracts. No public graph will have extracted from those. *Custom schemas* matter when the relationship types and entity distinctions that matter for your reasoning don't match what existing graphs provide. "Treats" and "associated with" are not interchangeable for a clinical decision support system. *Provenance requirements* matter when you need to trace every claim back to a specific source passage, with confidence and evidence type. Many public graphs aggregate without preserving that level of traceability. Building is not always the answer – but when one or more of these conditions holds, buying often isn't either.

An honest accounting. This book is about building. It would be

dishonest to pretend that building is always the right choice, or that the approach here is the only one. The goal is to give you the tools to make the tradeoff consciously: to know what you gain by building, what you give up, and when the calculus favors one path over the other.

What a KG Is Not Good For

A knowledge graph is a powerful tool for certain kinds of reasoning. It is not a general-purpose solution, and overclaiming its virtues does the field no favors.

Quality reflects sources. A knowledge graph encodes what is in its sources. If the sources are biased, incomplete, or wrong, the graph will be too. Extraction can introduce additional errors – misclassified relationships, wrong entity resolutions, spurious connections – but the ceiling is set by the corpus. A graph built from low-quality literature will not magically produce high-quality knowledge. Garbage in, garbage out applies with full force.

Coverage gaps are structural. A graph can only represent what has been extracted. If a domain is under-studied, or if the important relationships are stated in ways the extractor doesn't recognize, the graph will have holes. Those holes are not always obvious: absence of an edge can mean "no relationship" or "we haven't seen it yet." Reasoning over an incomplete graph can produce false negatives – "the graph doesn't show a connection" is not the same as "no connection exists." Users need to understand the difference.

Maintenance is ongoing. Knowledge decays. New papers are published, consensus shifts, drugs are approved or withdrawn, mechanisms are revised. A static graph becomes stale. Keeping it current requires continuous ingestion, schema evolution as the domain evolves, and curation to correct extraction errors and resolve conflicts. This is not a one-time build; it's an ongoing commitment. Organizations that treat a knowledge graph as a project rather than a product often find that the graph drifts out of usefulness within a year or two.

Bias encodes at scale. The literature in many domains reflects historical and structural biases: which diseases get studied, which

populations are represented in trials, which research questions receive funding. A graph extracted from that literature inherits those biases. Worse, the graph can amplify them – a pattern that appears in many papers becomes many edges, which makes it look more established than a pattern that appears in few. A knowledge graph is not neutral. It reflects the priorities and blind spots of its sources, and those need to be understood and, where possible, corrected.

Chapter 4: Representation Is Reasoning

From fluency to grounded reasoning

Chapter 1 established that LLMs are fluent but not grounded – they have no way to gauge correctness or signal uncertainty, and they fail unpredictably when we leave the training distribution. The consequence that matters here: a system reasoning from statistical patterns fails in ways that are opaque, while a system reasoning from explicit, structured knowledge about the domain fails in ways that are traceable, correctable, and bounded by what's in the representation. That distinction is what the rest of this chapter is about.

Experts have internal knowledge graphs

Here is the argument for knowledge graphs, stated plainly: genuine reasoning about a complex domain requires a representation that makes the structure of that domain explicit, inspectable, and correctable. Not as an engineering convenience. As an epistemological necessity.

But there's a version of this argument that undersells itself, and it's worth avoiding. The weak version says: machines need explicit knowledge representations because they can't do what humans do implicitly. The strong version – the one worth making – says: humans need explicit knowledge representations too, for exactly the same

reasons, and the best human expertise already has them, just not written down in a form that machines can use.

Think about what it means to be genuinely expert in a complex domain. A working cardiologist doesn't hold the relevant knowledge as a pile of facts. She holds it as a structured web of relationships – this drug potentiates that pathway, this symptom cluster suggests this differential, this interaction is dangerous in patients with this history. The knowledge is relational. It has direction. It has confidence levels, implicitly – she trusts the large randomized trials more than the case reports, the established mechanisms more than the preliminary findings. She has, in effect, a knowledge graph in her head, built over years of training and practice. What she doesn't have is an artifact that a machine can query.

The knowledge graph is not a substitute for that expertise. It's an attempt to make its structure explicit – to take the relational model that the expert has built and put it in a form that can be shared, extended, corrected, and reasoned over by systems that didn't spend fifteen years in medical training. The central argument shifts from "machines need this" to something more interesting: machines and humans are doing the same thing, and now we can make the shared structure visible.

This reframing has a consequence that matters for how you think about the future of the field. The objection that large language models are getting better fast – that the case for explicit knowledge representation is really just a case for not-yet-good-enough LLMs, and will dissolve as the models improve – misses the point. A more capable language model reasons better over its training distribution. It does not, by virtue of being larger or better trained, acquire the specific, curated, provenance-tracked model of *this* domain as *this* community of experts currently understands it. That model is constructed through human judgment, domain expertise, and deliberate curation. No amount of training data substitutes for it, because training data reflects the past and the general, while a curated knowledge graph reflects the present and the specific. The cardiologist's knowledge graph, if it existed and were kept current, would contain things that aren't in any published paper yet – the pattern she noticed last month, the contraindication that her department started flagging based on three recent cases, the consensus that has shifted

but hasn't been formally written up. Training data is always behind the frontier of expert knowledge. A living graph doesn't have to be.

Grounded representation as the fix

Chapter 1 argued that hallucination is baked in, not a bug. The fix is to give the model something to reason *from* – explicit, structured, checkable claims. A knowledge graph does that: the model is shown edges, sources, and confidence, not asked to retrieve from statistical memory. That's a different cognitive task, and it produces different results.

Provenance, auditability, trust

A knowledge graph is a model of a domain, not the domain itself. This distinction sounds pedantic until you think about what it implies.

The implicit "model" inside a neural network is also a model of a domain – or rather, of many domains simultaneously, encoded in weights that are not directly interpretable. It cannot be inspected. You cannot ask the model to show you its representation of the relationship between a drug and its target protein. You cannot correct it when that representation is wrong. You cannot extend it with new knowledge without retraining. You cannot audit it for bias or gaps. The model is a black box with a surface – you can probe the surface, but the interior is not accessible.

An explicit representation – a knowledge graph – is a different kind of thing. It can be inspected. Every entity can be examined, every relationship can be queried, every provenance record can be traced back to its source. When it's wrong, it can be corrected. When the domain changes, it can be updated. When you want to know why the system believes something, you can follow the chain of evidence. Auditability is not just a nice property – in any domain where the reasoning matters, it is a requirement. A physician using an AI system to inform a treatment decision needs to be able to ask "why" and get an answer that makes sense. A lawyer relying on an AI-assisted analysis needs to be able to trace the claim to its source. An explicit representation makes this possible. An implicit one doesn't.

The history of knowledge representation in AI is, in one reading, a long argument about this distinction. The expert systems of the 1980s had it right in principle: they reasoned over explicit representations, their inferences were in principle auditable, and when they were wrong you could usually figure out why. What they got wrong was the economics: building and maintaining those representations required armies of knowledge engineers working with domain experts, and it didn't scale. The logic-based systems were brittle because the representations were brittle – incomplete, inconsistent, and expensive to update. The statistical revolution of the 1990s and 2000s threw out the explicit representation in favor of learned, implicit ones, and gained enormous practical capability at the cost of auditability. The current moment is the first time in the history of the field that it has been practically possible to build explicit, structured, domain-specific representations at scale without armies of knowledge engineers – because the extraction step, the part that was always the bottleneck, can now be done by a language model with a well-designed prompt.

The argument comes full circle. Representation matters. Explicit representation matters more than implicit representation in any domain where the reasoning has to be auditable. Extraction is what makes explicit representation tractable at scale. Language models provide the extraction. The rest of this book is the engineering.

Chapter 5: The Extraction Problem

Humans are smarter than you think

Consider this sentence, from a real paper in the cancer literature:

> Patients treated with the combination showed significantly reduced tumor burden compared to controls, though the effect was attenuated in those with prior platinum exposure.

Read it once and you already know, if you have any background in the domain, roughly what it's saying. There's a treatment – a combination of something, referenced earlier in the paper – that works against tumor growth. The evidence is significant, which means it cleared a statistical threshold. But the effect is weaker – attenuated – in patients who have previously received platinum-based chemotherapy. This is a clinically important qualification: prior platinum exposure is a common history in many cancer populations, so "works, but less well if you've had platinum" is a materially different clinical claim from "works".

Fifty words. A finding, a population, a comparison structure, a statistical hedge, a subgroup qualification, and an implicit clinical contraindication. A human reader with domain knowledge unpacks all of this in roughly the time it takes to read it once.

Now ask what it would take for a machine to do the same.

The finding itself is not stated as a simple subject-verb-object. "Patients treated with the combination" is the subject, but the combi-

nation is not named here – its identity requires reading earlier in the paper, which requires co-reference resolution across sentence boundaries. "Showed significantly reduced tumor burden" is the claim, but "significantly reduced" is a statistical characterization, not a raw observation, and "tumor burden" is a clinical measurement that needs to be recognized as such and linked to its standard definition. "Compared to controls" establishes the comparison structure – this isn't an absolute claim, it's a relative one, and losing that distinction changes the meaning. "Though the effect was attenuated" introduces hedging – not a negation, but a qualification. And "prior platinum exposure" names a variable that modulates the effect, which means the machine needs to understand not just that platinum is a drug, but that prior exposure to it is a patient characteristic that interacts with treatment response.

This is not an unusually complex sentence for the biomedical literature. It's representative. And the extraction problem is the problem of reading sentences like this, millions of them, across thousands of papers, and producing structured, typed, provenance-tracked knowledge from them reliably enough to be useful.

NLP was insufficient to the task

It is worth being honest about what the field of natural language processing actually achieved before large language models arrived, because the temptation to either overstate or dismiss that progress is real.

Named entity recognition – NER – had become genuinely practical by the mid-2010s. Systems trained on annotated biomedical corpora could identify genes, diseases, drugs, and chemicals in text with accuracy that was useful for downstream applications. The BioBERT family of models, pre-trained on PubMed abstracts and fine-tuned for specific tasks, set benchmarks that were hard to dismiss. Co-reference resolution – the problem of knowing that "the compound" in one sentence refers to "imatinib" in the previous one – made real progress, though it remained brittle on the long-range dependencies that appear routinely in scientific prose. Relation extraction – identifying that two named entities stand in a specific relationship – worked well in narrow domains with sufficient training data and carefully de-

fined relationship types.

These weren't failures. They were genuine scientific and engineering progress, and the systems built on them were in production at pharmaceutical companies, biomedical literature services, and research institutions. The field knew what it was doing.

The brittleness showed up at the edges, and the edges were everywhere.

Domain adaptation was the first wall. A relation extraction system trained on biomedical literature needed to be substantially retrained to work on legal documents. The vocabulary was different, the sentence structures were different, the implicit conventions about how claims were stated were different. This wasn't a matter of fine-tuning a few parameters – it was, in practice, a research project. You needed new training data, which meant new annotation, which meant hiring domain experts and building annotation pipelines and managing annotator disagreement. The cycle time from "we want to build a KG in this new domain" to "we have a working extraction pipeline" was measured in months, and the result was never quite as good as you hoped.

The annotation treadmill was the second wall, and it interacted badly with the first. Supervised extraction requires labeled data. Labeled data requires human judgment. Human judgment is expensive, inconsistent, and always slightly out of date. Domain experts disagree about edge cases – and in complex domains, there are a lot of edge cases. Schemas evolve as understanding improves, which means last year's annotations are partially wrong for this year's schema. The pipeline you trained for the schema you had in January doesn't quite fit the schema you have in June. You annotate more data. You retrain. The schema changes again. The treadmill is always moving.

There was also something more fundamental. Classical NLP worked by learning statistical proxies for semantic relationships – patterns of words, grammatical structures, co-occurrence statistics that correlated with the relationships you cared about. This worked well when the patterns were consistent and the training data was representative. It worked poorly on hedged language, because "did not inhibit" and "inhibits" have very similar statistical fingerprints but opposite meanings. It worked poorly on implicit relationships – the kind where

the text doesn't state the relationship directly but a knowledgeable reader infers it. It worked poorly on domain jargon that appeared rarely enough in training data to be statistically invisible. And it worked poorly, structurally, on anything that required integrating information across multiple sentences or multiple documents to establish a single relationship, because most classical architectures had no mechanism for that kind of extended context.

The honest summary: classical NLP built extractors that worked well on the easy cases, degraded gracefully on the medium cases, and failed in ways that were hard to characterize on the hard cases. For many applications, "works well on the easy cases" was sufficient. For building a knowledge graph from a large, diverse scientific literature, it wasn't.

Economics of LLMs for extraction

The shift that large language models bring is not, primarily, a capability shift. It's an economic one, and the economic shift is what makes the capability shift matter.

The marginal cost of a new extraction task, with classical NLP, was a research project: weeks or months of work, annotated training data, iterative refinement against a domain-specific benchmark, deployment engineering, and ongoing maintenance as the schema evolved. For organizations with the resources to do this, it was feasible. For everyone else, it wasn't. Knowledge graph construction from unstructured text was, in practice, a capability available only to well-resourced actors in high-value domains.

The marginal cost of a new extraction task with a large language model is a prompt. Not a simple prompt – a good extraction prompt takes thought, domain knowledge, and iterative refinement, and we'll spend a chapter on how to write one. But the cycle from "I want to extract this kind of relationship" to "I have a working extractor" is measured in hours, not months. Schema changes don't require retraining. A domain expert who can't write code can participate meaningfully in defining what the extraction should look for. The feedback loop between schema design and extraction quality – always important, usually expensive to close – becomes something you can iterate through in an afternoon.

This is a phase transition, not an incremental improvement. When the cost of a capability drops by two orders of magnitude, the set of people who can use it changes qualitatively, not just quantitatively. Knowledge graph construction from unstructured text is no longer a capability for organizations with research teams and annotation budgets. It's a capability for anyone with a corpus, a clear schema, and the patience to iterate on prompts.

What LLMs bring to the specific challenges of extraction is also worth being concrete about. Hedging and negation – the bane of classical systems – are handled naturally by a model that has learned from an enormous amount of human language, most of which contains hedging and negation. Implicit relationships – the ones a knowledgeable reader infers rather than reads directly – are within reach of a model with enough domain knowledge in its training distribution. Cross-sentence dependencies, which defeated most classical architectures, are handled by the attention mechanisms that are foundational to the transformer architecture. Domain jargon is less of a problem when the model has been trained on a corpus large enough to have seen most of it.

None of this is magic, and it's worth being precise about what it isn't.

Hallucination (Chapter 1) takes a specific form in extraction: the model can invent entity names that don't appear in the source, fabricate relationships the text doesn't assert, and misattribute provenance. Validation is not optional.

Context window limits matter for scientific literature, where a paper is often tens of thousands of words and the relationships that matter may span sections written pages apart. The chunking strategies in Chapter 10 are a response to this reality. They work, but they introduce their own complications: a relationship that spans a chunk boundary may be missed, and the context available to the model for any given extraction is always less than the full paper.

Cost at serious scale is real. Extracting a comprehensive knowledge graph from a large corpus – hundreds of thousands of papers – requires a large number of API calls, and API calls are not free. The economics are better than classical NLP at the research prototype scale and require more careful management at production scale.

And non-determinism – the fact that the same prompt, run twice against the same text, may produce slightly different output – has implications for reproducibility that any serious pipeline needs to address. Caching extraction results is not just an efficiency measure; it's a reproducibility measure.

This chapter ends honestly because the rest of the book is the engineering response to these limitations. LLMs are the best tool we have ever had for the extraction problem. That's a strong claim and the evidence for it is in the following chapters. But "best we've ever had" and "good enough to use without careful engineering" are not the same thing, and conflating them leads to pipelines that work in demos and break on real corpora.

Part II: LLMs Change the Equation

Chapter 6: LLMs Make This Practical Now

The first five chapters of this book made two kinds of arguments. One was conceptual.

- Knowledge graphs are necessary for reliable machine reasoning, not just convenient.
- Explicit representation is epistemologically superior to implicit representation in any domain where the reasoning has to be auditable.
- The structure of expert knowledge is relational, and a graph is the natural data structure for capturing it.

The other argument was historical.

- People have been trying to build these things for decades.
- The conceptual case was understood early.
- The work kept stalling at the same bottleneck.

Getting knowledge *in* – turning unstructured text into structured, typed, provenance-tracked graph data at scale – was the hard part, and it stayed hard for a long time. This chapter is about why it stopped being hard.

The answer is large language models, but the reason they matter for this problem is not primarily about capability. It's about economics. The capability is real and we'll get to it. But capability arguments for AI systems tend to slide into hype, and the hype obscures what actually changed. What actually changed is that the cost structure of knowledge graph construction from unstructured text underwent a phase transition – from "research project" to "afternoon of prompt

engineering" – and that change in cost is what changed who can build these things and what they can be built for.

The Economics Argument First

Before we talk about what LLMs can do, let's talk about what classical NLP couldn't afford to do.

Chapter 5 walked through the classical extraction pipeline and its limitations. The technical limitations were real, but they weren't the main barrier. The main barrier was the economics of domain adaptation. If you wanted to extract knowledge from biomedical literature, you needed an extraction system tuned to biomedical literature: trained on annotated biomedical data, evaluated against biomedical benchmarks, maintained as the schema evolved and the literature changed. If you then wanted to do the same for legal documents, you were starting over. Different vocabulary, different sentence structures, different implicit conventions about how claims are stated. A new training dataset. A new annotation effort. New domain experts, recruited, trained, and calibrated. A new deployment pipeline.

This wasn't a failure of the technology. It was the technology working as designed. Supervised machine learning systems are trained on distributions, and they work on those distributions. Adapting them to new distributions is a research problem. For well-resourced organizations with stable, high-value domains – pharmaceutical companies mining drug interaction literature, financial institutions processing regulatory filings – the investment made sense. The organizations that couldn't make that investment simply didn't build knowledge graphs from their unstructured text, because the alternative was too expensive.

The practical consequence was that knowledge graph construction from literature was a capability concentrated in a small number of well-resourced actors. Everyone else either used the knowledge graphs those actors made available, or went without.

What changes with large language models is the marginal cost of a new extraction task.

Not zero – there's no free lunch, and we'll catalog the costs honestly

in a moment. But the order of magnitude shifts dramatically. A classical NLP system adapted to a new domain required months of work from a team with specialized skills. An LLM-based extraction pipeline for a new domain requires a prompt that describes what you're looking for. A good prompt takes thought, iteration, and domain knowledge; we'll spend Chapter 10 on how to write one. The cycle from "I want to extract this kind of relationship" to "I have a working extractor" is measured in hours or days, not months. Schema changes don't require retraining. A domain expert who can't write code can look at a prompt, understand what it's asking for, and suggest improvements. The feedback loop between schema design and extraction quality – always important, almost always expensive to close in classical systems – becomes something you can iterate through in an afternoon.

This is a phase transition, not an incremental improvement. When the cost of a capability drops by two orders of magnitude, the population of people and organizations who can use it changes qualitatively. Knowledge graph construction from unstructured text is no longer a capability reserved for organizations with research teams and annotation budgets. It's available to anyone with a corpus, a clear schema, and the patience to iterate.

That matters for who builds knowledge graphs, what domains get covered, and ultimately what gets done with them. The broader implications are in Part IV. For now, the point is narrower: the barrier that kept this technology concentrated in a few hands has lowered. The rest of this chapter is about why the lowering is real and durable, and where the remaining barriers are.

What LLMs Actually Are, For This Purpose

Chapter 1 established what LLMs are not: not databases, not reasoning systems, not reliable reporters of ground truth. They are pattern-completion engines trained on large text corpora, and their outputs are statistically plausible continuations of their inputs, not verified facts. This is the source of hallucination, and hallucination does not go away in the extraction context – it takes a specific form there that we'll address directly.

What LLMs *are*, for the purposes of extraction, is something more

specific and more useful than "a neural network trained on text."

A large language model has, in its training distribution, an enormous amount of human text that includes natural language descriptions of relationships between things: what drugs treat, what genes encode, what historical events caused other historical events, what legal provisions imply, what symptoms suggest. The model has learned, in a diffuse statistical sense, what it means for these relationships to hold – not as explicit logical rules, but as patterns of co-occurrence, context, and usage that are deeply embedded in the model's weights.

When you write a prompt that says "extract all 'treats' relationships between drugs and diseases from the following text, where 'treats' means a drug is used therapeutically to address a disease," you are not teaching the model what "treats" means. You are binding the model's existing, diffuse understanding of that concept to your schema. The model already has a representation of the difference between "ibuprofen treats headache" and "ibuprofen does not treat bacterial infection." Your prompt tells it that this particular distinction, expressed in terms of your schema's relationship type, is what you want surfaced.

This is what the outline calls "the prompt as schema binding," and it's the conceptual key to why LLM-based extraction works differently from classical systems. You're not training a model to recognize patterns. You're directing a model that already has deep, broad pattern knowledge to apply that knowledge to your specific representational task. The schema description in your prompt is a set of instructions to a very knowledgeable reader, not a specification for a statistical classifier.

The implications of this are large, and we'll work through them in the next section. For now, the key point is that "LLMs understand language" – a claim that deserves skepticism in many contexts – is, for the specific task of extraction, a practically useful approximation. The model doesn't need to understand language in the philosophical sense. It needs to be able to identify, in a passage of text, whether a relationship of a given semantic type holds between two given entities. It can do this because it has learned, from an enormous amount of human language use, what those relationships look like when they're present and when they're absent. That's enough.

The Prompt as Schema Binding

The central practical difference between classical NLP extraction and LLM-based extraction is this: in classical systems, the schema is baked into the model architecture and the training data. In LLM-based systems, the schema is in the prompt.

This sounds like an implementation detail. It isn't.

When the schema is baked into the model, changing the schema means changing the model. New entity types require new training data and new training runs. Renamed relationship types confuse a model trained with the old names. Distinctions that turned out to matter – the difference between "directly inhibits" and "allosterically inhibits," say, if that distinction turns out to be clinically significant for your application – require new annotations and new training. The schema is frozen at training time, and thawing it is expensive.

When the schema is in the prompt, changing the schema means changing the prompt. You add a new entity type by describing it. You clarify a relationship type by adding a sentence that explains the distinction. You can make these changes and run an extraction batch within the hour to see how the change affects output quality. Schema evolution, which any serious knowledge graph project will go through, stops being a recurring research project and becomes a routine iteration loop.

The other consequence is that schema design becomes a collaborative, legible activity. A classical NLP pipeline encodes schema decisions in training data and model weights that domain experts can't directly inspect or critique. A prompt encodes schema decisions in natural language that anyone who can read can evaluate. A cardiologist reviewing a proposed schema for a cardiovascular knowledge graph can read a well-written extraction prompt, notice that the distinction between "increases risk of" and "causes" is not being drawn, and say so. She doesn't need to understand machine learning. She needs to understand her domain, and she does.

This changes the knowledge engineering relationship in a way that the classical systems never managed. The expert systems of the 1980s aspired to this: the vision was always that domain experts would be able to inspect and correct the knowledge base. The execution re-

quired knowledge engineers as intermediaries, because the representation languages were not natural language. LLM-based extraction achieves, in the extraction domain, what the knowledge engineers were supposed to achieve in the representation domain: it removes the translation layer between what domain experts know and what the system can use.

There are limits. Prompts that ask for overly subtle distinctions – semantic differences that would challenge even a careful human reader – produce inconsistent output. Prompts that are ambiguous produce ambiguous extractions. The quality of the schema description in the prompt directly determines the quality of the extraction output, and "write a good schema description" is harder than it sounds; the whole of Chapter 10 is devoted to it. The point is not that prompts are easy to write, but that the feedback loop between "what I described" and "what I got" is short enough to be useful.

Handling What Classical Systems Couldn't

Let's be concrete about the specific failure modes of classical NLP extraction that LLMs handle well, because "LLMs are better at language" is too vague to be useful.

Hedging and negation. The sentence "drug X did not inhibit pathway Y in this model" contains an "inhibits" relationship that is negated. A classical relation extraction system trained to recognize inhibition relationships would frequently fire on this sentence anyway, because "did not inhibit" and "inhibits" look statistically similar. The word "not" is short and common enough to be underweighted in most feature representations. A large language model, prompted to extract inhibition relationships and to exclude negated ones, handles this correctly in the vast majority of cases. The model has learned from an enormous amount of text that negation changes meaning, and it applies that knowledge. This is not a small improvement: negated relationships are common in scientific literature, and a graph that includes them as positive edges is systematically wrong in ways that are hard to detect.

Hedged claims. Related but distinct. "Drug X may inhibit pathway Y" and "drug X inhibits pathway Y" are different claims with different epistemic weights. Classical systems often collapsed these

into the same extraction, losing the hedge. A prompted LLM can be instructed to track hedging as part of the provenance record – to note whether a claim is stated as fact, hypothesis, or speculation – and it will do so with reasonable consistency. This is directly relevant to provenance design, which Chapter 8 covers.

Implicit relationships. Some relationships are never stated directly in the text but are clear to a knowledgeable reader. "Patients receiving drug X showed a 40% reduction in tumor burden compared to controls" does not contain the word "treats." It asserts, in the language of clinical reporting, that drug X has therapeutic activity against the relevant tumor type. A classical system without an explicit "treats" pattern for this construction would miss it. A large language model, prompted to extract treatment relationships and given a description of what counts as evidence for one, will recognize this as an instance of the pattern. This matters a great deal in biomedical literature, where direct assertions are often replaced by results-focused constructions that any domain expert reads as making a relational claim.

Cross-sentence dependencies. "The compound was tested against a panel of cancer cell lines. It showed selective activity against BRCA1-mutant cells, with IC50 values in the nanomolar range." These two sentences together assert a relationship – the compound has activity against a specific molecular subtype of cancer – that doesn't fully exist in either sentence alone. Classical architectures with limited context windows would process each sentence independently and might miss the connection. A large language model operating over both sentences simultaneously – or, in the chunking strategies we'll cover in Chapter 10, over a passage that includes both – will recognize the implicit antecedent and extract the relationship correctly.

Domain jargon. A drug referred to by a trial identifier, a gene referred to by a lab-specific shorthand, a syndrome referred to by the name of its first describer – these appear constantly in specialist literature and were frequently invisible to classical systems trained on corpora where the standard terminology dominated. LLMs trained on broad scientific text have seen a much wider range of how concepts are referred to, and they tolerate terminological variation better. This isn't complete – genuinely novel terminology, or highly

specialized jargon outside the training distribution, can still cause failures – but the robustness is substantially better.

None of these improvements mean that LLM extraction is reliable without engineering. They mean that the specific failure modes that made classical extraction brittle in complex domains are substantially mitigated. The failure modes that remain are different in character, and the engineering response to them is different.

The Remaining Limitations, Honestly

Chapter 1 established hallucination as a structural feature of LLMs, not a bug. Chapter 5 was honest about what classical NLP couldn't do. This chapter should be equally honest about what LLMs can't do, because the engineering in Part III is largely a response to these limitations.

Hallucination in extraction. The model can invent entity names that don't appear in the source text. It can assert relationships that the text implies but doesn't actually support. It can misattribute provenance, assigning a claim to a passage that doesn't contain it. These are not rare edge cases – they occur with meaningful frequency even in well-prompted models, and the frequency increases as the task gets harder (more complex sentences, more ambiguous relationships, longer passages). Validation against the source is not optional. Chapter 12 covers the validation pipeline.

Context window limits. Scientific papers are often tens of thousands of words. The relationships that matter may span sections written pages apart – an introduction that defines a hypothesis, a methods section that describes a test, a results section that reports an outcome. The model's context window is finite, and even the largest current context windows don't fully solve this problem; performance tends to degrade on information that appears far from the relevant extraction target within a long context. The chunking strategies in Chapter 10 are a pragmatic response: break documents into manageable passages, extract relationships within each, and handle cross-chunk dependencies with a separate pass. This works, but it introduces its own complications, including relationships that span chunk boundaries and may be missed.

Cost at scale. A single extraction call against a single paper costs a small amount. Across a corpus of hundreds of thousands of papers, the cost accumulates. The economics are better than classical NLP at the prototype scale – dramatically so – and require more careful management at production scale. Caching, batching, and tiered extraction (do cheap passes first, expensive passes only where needed) are all part of managing this.

Non-determinism. The same prompt, run twice against the same text, may produce different output. This matters for reproducibility: if your pipeline produces different graphs on different runs, it's difficult to debug, compare, and maintain. Caching extraction results addresses this directly and is both an efficiency measure and a reproducibility measure.

What the model knows and doesn't know. LLM extraction is anchored in the model's training distribution. Very recent developments, highly specialized terminology that appears rarely in broad scientific text, and concepts that are standard in one community but not in the general scientific literature can all cause degraded performance. In practice, this means that extraction quality should be evaluated on your specific domain and corpus, not assumed from general benchmarks.

The point of this honest accounting is not to undercut the argument that LLMs make knowledge graph construction newly practical. They do. The point is that "newly practical" means "practical if you engineer it carefully," not "practical if you just call the API." Part III is the careful engineering.

Why This Moment

One more question deserves an answer before we get into the engineering: why now? The transformer architecture was introduced in 2017. GPT-2 was released in 2019. Why is this moment – roughly 2023 through the present – the right time to build?

The answer is the convergence of three things that had to arrive together.

The first is model capability. The capability of general-purpose language models for complex semantic understanding crossed a thresh-

old somewhere around the GPT-4 generation. Before that threshold, prompted extraction was possible but brittle on complex constructions; after it, the failures became manageable with good engineering. Earlier models were impressive but required more hand-holding. Current models handle the hard cases – hedging, negation, implicit relationships, cross-sentence dependencies – with the consistency needed for production pipelines.

The second is API accessibility. Using a capable language model for extraction before the current generation of public APIs meant running your own infrastructure, which meant GPU clusters, model serving, and the full operational stack. This was possible for large organizations; it wasn't feasible for a researcher, a small company, or an individual practitioner. The existence of stable, affordable, well-documented APIs for capable models changes who can build this. You don't need infrastructure. You need an API key and a credit card.

The third is the tooling ecosystem. The infrastructure for building knowledge graph pipelines – graph databases with native support for the relevant query patterns, vector databases for embedding-based retrieval, orchestration frameworks for multi-step LLM pipelines – arrived, matured, and became accessible at roughly the same time as the models. A knowledge graph pipeline in 2020 would have required assembling a stack of immature tools and writing substantial infrastructure code. Today the stack exists and the components are well-documented.

These three things had to be true simultaneously, and they are now. That matters for the argument this book is making. This is not a forecast that something will soon be possible. This is a description of what is possible today, and the rest of the book is how to do it.

There is also a fourth consideration that is more speculative but worth naming. We are at an early moment in the adoption of this capability. The knowledge graphs that will have the largest impact on medicine, law, materials science, and other complex domains don't exist yet. The tooling is new enough that best practices are still being established. The organizations that will benefit most from these systems are still figuring out that this is possible. The researchers who will do the most interesting work with these systems haven't started yet.

Early maps of large territories are valuable precisely because they're early. What follows in this book is one such map, drawn from working code and real corpora. It's not complete. It's not the last word. But the territory is real, the tools are here, and the problems are interesting.

Chapter 7: The Free KG Cases

Not every knowledge graph requires extraction. Some are built from sources that are already structured when they arrive – from lab instruments, databases, ontologies, or human-curated encyclopedias. Understanding these "free" cases sharpens the argument for extraction by making clear what the hard problem actually is. It also sets a quality benchmark: the graphs built without extraction tend to be high-precision over their coverage, precisely because the structure was already there. The extraction problem is the problem of accessing the knowledge that *wasn't* structured – and in most interesting domains, that's the majority of what's known.

When You Don't Need Extraction

The central claim of this book is that extraction from unstructured text is the bottleneck that has limited knowledge graphs for decades, and that LLMs have finally made it tractable. But that claim only applies when the knowledge lives in unstructured form. In many situations, the knowledge is already structured when it reaches you.

Perhaps the data comes from lab equipment that outputs well-defined records – a sequencer, a spectrometer, a sensor network. Perhaps it comes from a database table with columns and foreign keys. Perhaps it comes from an ontology or formal specification that was designed to be machine-readable from the start. In these cases, the mapping from source to graph is often straightforward: a short script, an ETL pipeline, or a direct translation of the source schema into graph form. No LLM is required. No extraction prompt. No ambiguity

about what the text meant. The structure is given.

This chapter examines several such cases. The goal is not to dismiss them – they are important and widely used – but to clarify the boundary. When does a knowledge graph require extraction, and when does it not? The answer determines whether the techniques in the rest of this book apply to a reader's problem, and it also illuminates what extraction is *for*: bridging the gap between the knowledge that is already structured and the knowledge that is not.

Lab Instruments and Measured Data

Genomics provides the canonical example. DNA sequencers, mass spectrometers, and high-throughput assay platforms produce structured data almost by definition. A sequencer outputs base calls with quality scores; a protein-protein interaction assay outputs pairs of identifiers with confidence metrics. The data has a schema. It has identifiers that map to established ontologies. It can be loaded into a graph with minimal interpretation.

Graphs like STRING (protein-protein interactions), BioGRID (genetic and physical interactions), and IntAct (molecular interactions) are populated primarily from such experimental measurements. They aggregate data from thousands of published studies, but the aggregation is over structured datasets that the authors deposited in repositories – not over the free text of the papers. The extraction problem, in this context, is largely solved by the experimental design: the scientist produces structured output, and the graph ingests it.

What these graphs are good for is clear. They support queries like "what proteins interact with BRCA1?" or "what pathways involve this gene?" with high precision, because the edges correspond to actual experimental observations. What they miss is equally important: the experiment that wasn't done, or wasn't published as structured data, or was described only in the discussion section of a paper, is not in the graph. The knowledge that lives in the prose – the mechanistic interpretation, the caveats, the connections to other domains that the authors didn't formalize – remains inaccessible. For many biomedical questions, that's the majority of what's known. Lab instruments and measured data give you a high-quality graph over a

subset of the domain. Extraction is what gives you access to the rest.

Generated and Synthetic Graphs

A related class of knowledge graphs is constructed from databases, ontologies, or formal specifications rather than from text or instruments. The Gene Ontology is a curated hierarchy of molecular function, biological process, and cellular component terms – it is a graph by design, with typed relationships (is_a, part_of, regulates) between terms. Drug interaction databases like DrugBank or STITCH map drugs to targets, indications, and interactions using structured records. Legal code can be represented as a graph of sections, references, and amendments. In each case, the source was already formal enough that conversion to graph form is a matter of schema mapping, not interpretation.

The precision of these graphs is high. When a relationship is asserted in the Gene Ontology, it has been reviewed by curators and validated against evidence. When a drug-target interaction appears in DrugBank, it has been extracted and verified by the database maintainers. The boundary between a knowledge graph and a very well-structured relational database starts to blur here – and that blurriness is instructive. A graph is often just a different view of the same data, with traversal and path-finding as the primary operations instead of joins and aggregations.

The limitation is coverage. These graphs contain what was explicitly encoded. They do not contain what was left implicit, or what was stated in natural language and never formalized, or what was discovered after the last curation pass. For domains where the authoritative knowledge is already in structured form, that may be sufficient. For domains where the knowledge lives in the literature, it is not.

Curated Graphs at Scale

Wikidata, Freebase before it, and DBpedia represent a different model: human curation at scale. Millions of entities, millions of relationships, maintained by a community of contributors who add facts, correct errors, and resolve disputes through discussion and consensus. The result is a graph that spans many domains, with

reasonable quality where the community has focused effort, and gaps where it has not.

What this achieves is remarkable. A single query can retrieve structured information about a person, a place, a chemical, a historical event – with identifiers that are stable, with relationships that are typed, with provenance that points to sources. For many applications, that is enough. The cost is the cost of human labor: curation does not scale to the full breadth of human knowledge, and it scales least well to domains where the knowledge is technical, specialized, or rapidly evolving. Encyclopedia articles can be curated. The full text of the medical literature cannot.

DBpedia illustrates the boundary clearly. It is extracted from Wikipedia infoboxes and structured elements – but the extraction is from semi-structured templates, not from free prose. The infobox for a drug might have a "mechanism" field; the body of the article might have three paragraphs of nuanced explanation that never made it into the template. DBpedia has the former. It does not have the latter. Curated graphs at scale work where the community can structure the knowledge. They do not work where the knowledge lives in papers, reports, and documents that no one has the bandwidth to formalize.

What These Cases Teach Us

The common thread across lab instruments, generated graphs, and curated encyclopedias is this: structured sources give you high-precision graphs over the knowledge that was *already structured*. The extraction problem is precisely the problem of accessing the knowledge that *wasn't* – which, in most interesting domains, is the majority of what's known.

The free KG cases set a quality benchmark worth aiming for. When a graph is built from structured sources, the edges tend to be correct, the entities tend to be well-identified, and the schema tends to be coherent. An extraction-based graph should aspire to that level of precision where it can. The gap between that benchmark and what extraction typically achieves is the gap that schema design, prompt engineering, and pipeline architecture are trying to close.

The free cases also illustrate the shape of the gap. It is not that extraction is impossible or that extracted graphs are inherently low quality. It is that extraction is a different kind of problem – one that requires interpreting natural language, resolving ambiguity, and making judgments about what the text implies. The tools for that problem have improved dramatically. The problem itself has not gone away.

Hybrid Approaches

Most real knowledge graphs combine extraction with structured sources. The typical pattern: extract entities and relationships from text, then link the extracted entities to authoritative identifiers from a curated database or ontology. A drug mention in a paper is resolved to an RxNorm code. A disease mention is resolved to a MeSH term. A gene mention is resolved to an HGNC identifier. The extraction provides the coverage – the connections that appear in the literature. The authority lookup provides the identity – the canonical form that makes those connections comparable across sources.

This hybrid approach does not make the extraction problem go away. It constrains it. Instead of inventing identifiers from scratch, the extractor (or a downstream resolution pass) maps to an existing vocabulary. That mapping is itself a form of extraction – the model must recognize that "ketoconazole" in the text refers to the same thing as RxNorm's concept for ketoconazole – but it is a narrower problem than inventing a full schema and populating it from scratch. The schema, or at least the entity vocabulary, is given by the authority. The extraction fills in the relationships and the provenance.

The medical literature example in Part III follows this pattern. Entities are extracted from papers and resolved to MeSH, HGNC, RxNorm, or provisional IDs when no authority match exists. The Gene Ontology and other ontologies provide a backbone for relationship types. The result is a graph that combines the coverage of extraction with the identity resolution of curated sources. It is not a free KG – extraction is central – but it is not a purely extracted KG either. The hybrid is the norm, and understanding both the extraction side and the structured side is necessary to build one

well.

Chapter 8: Designing Your Schema

This is the chapter that might surprise readers who came for the engineering. Schema design is not primarily a technical exercise. It is where decisions are made about what the domain *is* – what counts as an entity, what kinds of relationships matter, what level of granularity is useful. These are epistemological decisions, and they determine everything downstream. A poorly designed schema will produce a graph that cannot support the reasoning it was built for, no matter how good the extraction pipeline. A well-designed schema makes extraction easier, validation straightforward, and evolution manageable.

Schema Design as Intellectual Work

The temptation is to treat schema design as a matter of picking entity types and relationship names from a list – a checklist exercise that can be delegated or done quickly. That approach produces schemas that look reasonable on paper and fail in practice. The real work is understanding the domain well enough to know what distinctions matter for the questions the graph will answer.

Consider a medical literature knowledge graph. Should "dosage" be an entity type or a property of a relationship? If it's an entity, the graph can support queries like "what dosages of this drug have been studied?" and "how do recommended dosages vary across indications?" If it's a property, those queries become harder or impossible. The decision depends on what the graph is for. A graph built to support drug repurposing might not need dosage as an entity; a graph

built to support clinical decision support might need it badly. There is no universal right answer. The schema must reflect the use case.

The same applies to relationship types. "Inhibits" and "is associated with" are not interchangeable. One supports reasoning about mechanism; the other supports retrieval but not much else. The schema designer must decide how fine-grained the relationship vocabulary should be – fine enough to support the intended reasoning, coarse enough that extraction can reliably distinguish the types. That balance is domain-specific and use-case-specific. It cannot be looked up in a reference. It has to be thought through.

Entities: What Gets to Be a Thing

The question of what deserves a node in the graph is foundational. Genes, drugs, diseases, symptoms, procedures, proteins, pathways, patient populations, study designs – which of these are entities and which are properties of entities? The answer is neither obvious nor universal. It depends on the questions the graph is meant to answer.

A useful heuristic: an entity is something that can be referred to across multiple sources and that can participate in relationships as a subject or object. "Breast cancer" is an entity because it can be the subject of "is treated by" and the object of "targets." "50 mg" is typically a property – it describes a dosage, but it does not itself have relationships to other things in the same way. The boundary is not always sharp. "Cohort of postmenopausal women" might be an entity if the graph needs to reason about study populations; it might be a property of a study if the graph only needs to retrieve studies by population criteria.

The medical literature example makes specific choices: Disease, Drug, Gene, Protein, Symptom, Procedure, Publication, and a few others. Evidence is an entity – not just a property of a relationship, but a node with its own identity, because provenance needs to be queryable and traversable. The reasoning behind each choice is worth examining. Genes and proteins are both entities because the literature distinguishes them and the relationships differ – a drug might target a protein but regulate a gene. Symptoms are entities because they connect to diseases in ways that matter for differential diagnosis. The schema reflects the structure of the domain as it appears in

the sources.

Relationships: Meaning and Direction

The difference between "co-occurs with," "inhibits," "causes," and "is associated with" is enormous. Collapsing them into a single "related to" type produces a graph that can support retrieval but not reasoning. The graph can answer "what is connected to this drug?" but not "what does this drug inhibit?" or "what causes this symptom?" The relationship type carries the meaning. Lose the type, and you lose the ability to reason over it.

The tradeoff is between semantic precision and extraction recall. Relationship types that are too generic – "associated with" – are easy to extract and never wrong, but they are almost never useful. Relationship types that are too narrow – "allosterically inhibits" versus "competitively inhibits" – may be impossible for an extraction model to distinguish reliably. The right level is where the types are meaningfully distinct, expressible in natural language to the model, and actually present in the sources. That level varies by domain. It is discovered through iteration, not specified in advance.

Direction matters as well. "Drug A treats Disease B" and "Disease B is treated by Drug A" are the same fact, but in a directed graph they are different edges. Consistency across extraction runs and across sources requires that direction be part of the type definition – not a convention but a constraint. The schema should specify, for each relationship type, what the subject and object types are and which way the relationship points. That discipline prevents the subtle errors that accumulate when direction is left implicit.

Hierarchy and Inheritance

When should entity types form a hierarchy, and when is a flat list better? A hierarchy allows "Protein" to be a subtype of "Gene product," which allows queries over all gene products to include proteins. It also adds complexity: the extractor must decide whether a mention is a protein or a gene product, and the schema must define the inheritance rules. For some domains, that complexity pays off. For others, a flat list of types is simpler and sufficient.

The temptation to over-ontologize is real. Elaborate taxonomies look impressive and suggest rigor. They also make extraction harder – the model must choose among many similar types – and they can make the graph harder to query if the hierarchy is deep and the inheritance semantics are unclear. The principle: add hierarchy only when it supports reasoning that a flat schema cannot. If the main use case is "find all entities of type X," and X is a leaf in the hierarchy, the hierarchy may not be earning its keep.

Provenance as a First-Class Schema Concern

Chapter 3 argued that provenance is essential – that every claim in a knowledge graph needs to be traceable to its source, with confidence and evidence type. In schema design, that argument becomes concrete. How is provenance represented? Is it a property on each edge? A separate entity type? A linked structure of evidence nodes?

The choice affects everything downstream. If provenance is a property, it is easy to add but hard to query across sources. If it is an entity type, it becomes traversable: "show me all evidence for this relationship" becomes a graph query. The medical literature example uses evidence as an entity – each piece of evidence is a node with its own identity, linked to the relationship it supports. That design makes it possible to aggregate evidence across sources, to filter by confidence, and to detect when two sources conflict.

The fields that are often regretted later, when they were not included from the start: source document, section, passage offset, extraction method, confidence score, evidence type (e.g., RCT vs. case report). Adding them after the fact requires backfilling or accepting that older extractions lack them. Designing them in from the beginning avoids that debt.

Designing for Extraction

Schema choices affect how easy or hard extraction is. Some relationship types are natural to express in language – "treats," "causes," "inhibits" – and extraction models handle them well. Others are awkward – "has mechanism" or "exerts effect via" – and the model may struggle to recognize them consistently. Entity types that are

genuinely ambiguous even to a human reader – is "oxidative stress" a process or a condition? – will produce inconsistent extraction no matter how good the prompt.

The feedback loop between schema design and extraction quality is tight. A schema that is easy to extract from will produce better results. A schema that is hard to extract from will produce noise, and the natural response is to simplify the schema – which may sacrifice the reasoning capability the graph was built for. The alternative is to iterate on both together: adjust the schema when extraction consistently fails on a type, and adjust the extraction prompt when the schema is right but the model is not. Finalizing the schema before extraction begins is often a mistake. The schema and the extraction prompt should co-evolve.

Designing for Evolution

The schema will change. New entity types will be needed that were not anticipated. Relationship types will turn out to be too coarse or too fine. Distinctions that seemed unimportant will matter more than expected. The question is how to design so that evolution is manageable rather than catastrophic.

Versioning helps. When the schema changes, the extraction output format changes, and downstream consumers need to know which version they are looking at. A manifest or metadata that records the schema version with each bundle or each graph snapshot makes it possible to migrate incrementally and to reason about compatibility.

Migration is the hard part. Adding a new entity type is usually straightforward. Splitting an existing type, or changing the semantics of a relationship, can require backfilling or re-extraction. The argument for keeping the schema as simple as possible for as long as possible is practical: every entity type and relationship type is a commitment. Add only what is clearly needed. Defer the rest until the need is demonstrated.

A single source of truth for domain specifics pays off here. When entity types, predicates, and prompt instructions live in one module, a change in one place updates the extraction prompt, validation logic, and deduplication rules together. Splitting them across YAML con-

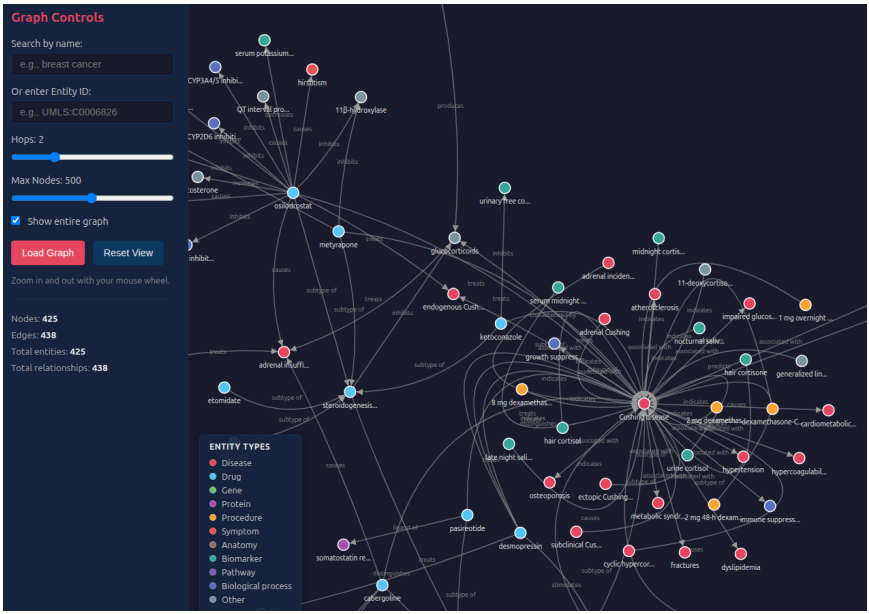
figs, Python code, and prompt templates invites drift: an entity type added in one file is forgotten in the prompt, or a predicate constraint tightened in the schema is not reflected in the dedup stage. One module, one edit, no drift. Readers building their own systems will find that maintaining a single domain specification module – whether in code, config, or a schema language – reduces the class of errors that come from inconsistent copies of the same information.

Part III: Building It

Chapter 9: Diagnostic Tools

The single most valuable diagnostic tool I have found is a good visualization view for the graph. Mine is done using the D3.js library's "force" example, where nodes and edges are treated like masses and springs floating in a 2-D space. A well-designed graph explorer does more than render nodes and edges – it gives you the controls to interrogate the graph and diagnose what's in it.

What can you learn from graph visualization as a diagnostic? Plenty. You can quickly determine whether you've got duplicate entities, whether entities that *should* be connected *are* connected as you'd expect, whether the graph is highly interconnected overall or is a collection of islands. All these are very useful hints about what's working, and what isn't, in how you generate your graph from the input data you accept.



Search by name or by canonical ID. The interface should support both. Searching by name – "breast cancer," "ketoconazole," "Cushing's disease" – is how you explore when you're thinking in domain terms. Searching by Entity ID – an HGNC symbol, an RxNorm code, a MeSH term – is how you verify that canonical identity is working. If you enter `MeSH:D000072716` and the graph loads the entity for "cancer" with its relationships, you've confirmed that your authority lookup and resolution pipeline are functioning. If the ID returns nothing, or returns the wrong entity, you've found a bug. The ability to query by canonical ID turns identity resolution from an opaque pipeline stage into something you can inspect directly.

Note on authority coverage: UMLS is the gold-standard disease authority for medical literature but requires a licensed API key. The medlit reference implementation uses MeSH as its primary disease authority (free, no key required) and falls back to DBPedia for concepts not covered by MeSH. Your domain may have different constraints; the diagnostic interface works the same way regardless of which authority IDs you assign.

Entity ID as a navigable link. When the detail panel shows an entity's canonical ID, that ID should link directly to the authority source – `MeSH:D000072716` links to the MeSH Browser, `HGNC:4556`

links to the HGNC gene page, `UniProt:P04637` links to the UniProt entry. This turns the entity panel from a dead display into a live bridge between your graph and the authoritative sources it draws from. A researcher who wants to verify a canonical assignment or read the full authority record can do so in one click. Provisional entities (those with internal `prov:` IDs, not yet resolved to any authority) display their ID as plain text.

Traversal controls: hops and node limits. A graph with thousands of entities is unreadable if you try to display it all at once. The diagnostic interface needs controls for scope. *Hops* – how many steps outward from the seed entity – determines how much of the neighborhood you see. One hop shows direct neighbors; two hops shows neighbors of neighbors, revealing indirect connections that might not be obvious from the raw data. *Max nodes* caps the display size for performance and readability. Together, these controls let you explore dense regions without overwhelming the renderer or your own perception. They also surface structural questions: if you expect an entity to have many connections but see few at two hops, either the graph is sparse in that region or your extraction missed relationships.

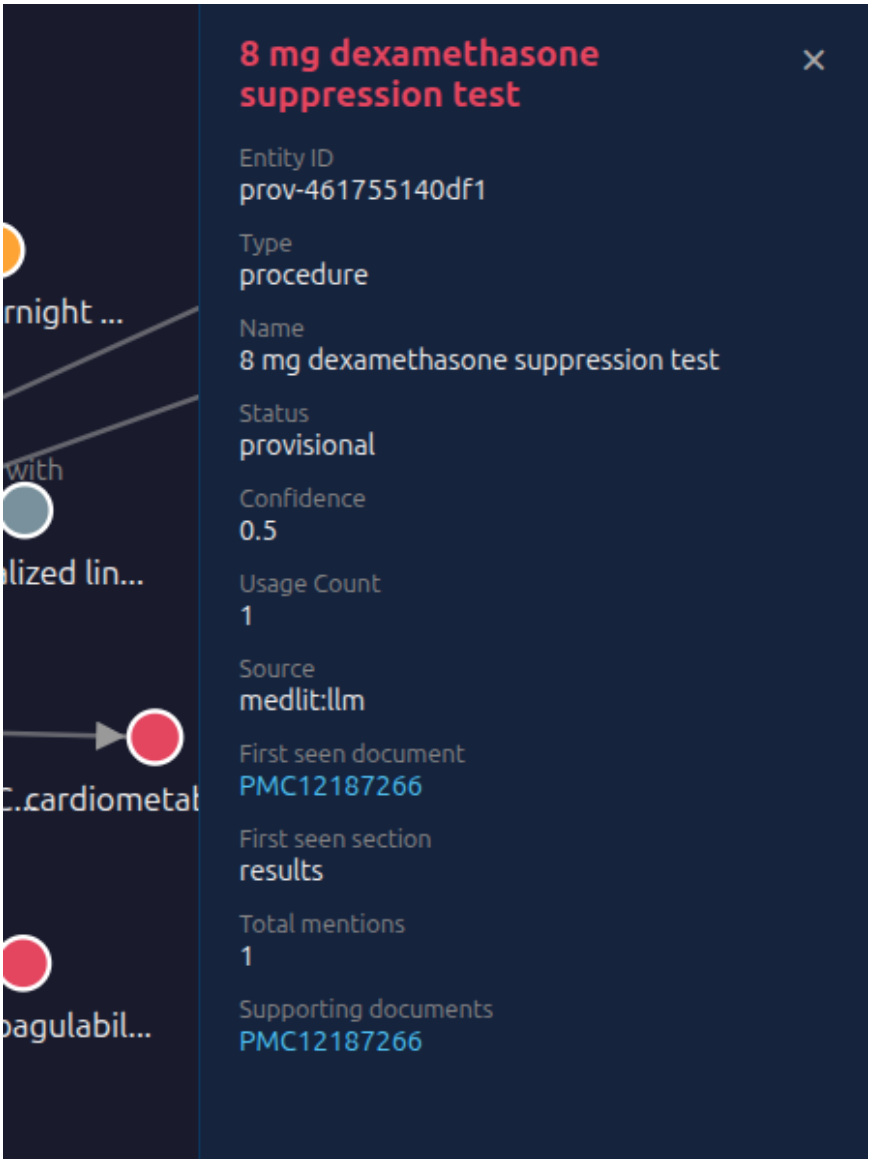
Entity type legend and color coding. Nodes should be colored by entity type – disease, drug, gene, protein, symptom, procedure, and so on. The legend makes the graph interpretable at a glance. You can see whether a subgraph is dominated by one type, whether the expected mix of types is present, and whether any node is misclassified. A gene colored as a disease, or a drug colored as a symptom, is an extraction error that becomes visible when the types are visually distinct.

Live statistics. Node count, edge count, total entities, total relationships – these numbers should update as you load different subgraphs. They give you an immediate sense of density. Is this entity well-connected or isolated? Does the neighborhood have the expected size given your corpus? Unexpectedly low counts can indicate extraction gaps or resolution failures; unexpectedly high counts can indicate over-merging or spurious relationship extraction.

Relationship labels on edges. The edges should show their types: "treats," "inhibits," "causes," "associated with," "subtype of." That visibility is diagnostic. You can spot relationships that are too generic ("associated with" everywhere suggests your extraction

prompt isn't distinguishing relationship types) or relationships that are wrong for the domain (a "treats" edge from a symptom to a disease is probably a mistake). The graph visualization becomes a quality audit: you're not just seeing that things connect, you're seeing *how* they connect, and that's where extraction errors show up.

More info on mouse-over and click. The specifics of what "more info" to display will depend on your application. For medical literature, I want to see what papers mentioned something, a level of confidence, any identifying information.



Zoom, pan, and reset complete the interface. The goal is to make the graph inspectable – to turn "does my ingestion process work?" into a question you can answer by looking.

Chapter 10: Design Priorities

This chapter examines the design decisions that matter most before you start building — specifically around provenance, which is the one you're most likely to underinvest in and most painful to retrofit. The examples are drawn from my own project, a knowledge graph to capture the contents of medical papers. I call this project "kgraph" in its domain-agnostic form, and "medlit" is the extension of the knowledge graph into the domain of medical literature.

These notes should be regarded not as a recipe or specification that you must follow, but rather as a checklist of things worth considering. The approach suggested here might meet your needs, it might not. Thinking about them will help you reason through the details of your own design.

Provenance

Provenance is not equally important in every domain. Be honest with yourself about how much provenance infrastructure your domain actually requires before you build it.

In medicine, the difference between "this drug inhibits this enzyme, from a single case report" and "this drug inhibits this enzyme, replicated across forty randomized controlled trials" is the difference between a hypothesis and a clinical fact. A clinician making a treatment decision needs to know which. A researcher generating hypotheses might care less about the evidence grade and more about the structural pattern. But for any application where the quality of the claim

matters – clinical decision support, regulatory submission, litigation – provenance is not optional. You need to know where each relationship came from, what kind of study supported it, and how confident the extraction was.

In other domains, the bar is lower. A knowledge graph of theatrical productions – which plays were performed where, when, by whom – might be perfectly useful without tracing every edge to a specific source. If the graph says "Hamlet was performed at the Globe in 1601," you might care that it's correct, but you probably don't need a provenance record that distinguishes "extracted from a primary source" versus "extracted from a secondary source" versus "inferred from context." The graph is useful even if you can't trace every edge. Building elaborate provenance infrastructure for a domain that doesn't need it is over-engineering.

The honest question: what will your users do with the graph? If they will make high-stakes decisions based on it, provenance matters. If they will use it for exploration, discovery, or casual reference, a lighter touch may suffice. Get that right before you invest in schema design and pipeline complexity.

When provenance does matter, it matters at an architectural level, not as an add-on. You can't retrofit provenance into a schema that didn't anticipate it without re-ingesting your corpus. The relationship model, the bundle structure, and the extraction prompts all need to capture provenance from the start. Think about it carefully before you build.

When it matters, it matters a lot. A relationship with provenance – specific source document, section or passage, extraction method, confidence score, study design if applicable – is evidence. Without provenance, a relationship in your graph is an assertion of unknown quality. You can't verify it. You can't weight it against conflicting claims. You can't explain to a user why the graph says what it says. You're left with "the graph says drug A treats disease B" and no way to assess whether that's a well-replicated finding or a single speculative mention.

Provenance is also what lets you audit extraction quality. When a relationship looks wrong, you need to trace it back: which document, which passage, what did the extractor see? That trace is how you

find prompt bugs, schema ambiguities, and source documents that are genuinely ambiguous. Without it, you're debugging in the dark.

It's also how you debug pipeline regressions. If extraction quality drops after a prompt change, provenance lets you compare: what did the old pipeline produce for this document versus the new one? Which relationships disappeared, which appeared, which changed? Provenance turns "something got worse" into "here's exactly what changed and where."

And it's how you answer the question every serious user will eventually ask: "Why does the graph say this?" A graph that can't answer that question is a black box. A graph that can produce, for any relationship, the list of sources that support it, with enough detail to verify, is a tool you can trust.

Confidence as a Signal, Not a Guarantee

LLMs can be prompted to produce confidence scores alongside their extractions. "How confident are you that this relationship holds, on a scale of 1 to 5?" The model will give you a number. That number is useful. It correlates with extraction quality: relationships the model is confident about tend to be more reliable than ones it's uncertain about. You can use it to filter, rank, or weight results. It's worth capturing.

It is not a calibrated probability. A model that says "90% confident" does not mean that 90% of such extractions will be correct. The model has no access to ground truth; it's producing a subjective assessment of its own certainty, which is a different thing. The scores are ordinal – higher usually means more reliable – but the mapping from score to actual accuracy is unknown and varies by domain, relationship type, and prompt.

The honest framing: confidence is one input to trust, not trust itself. A high-confidence extraction from a single case report is still weak evidence. A low-confidence extraction replicated across twenty randomized trials might be strong evidence. Confidence tells you something about the extraction process. Provenance tells you something about the underlying evidence. You need both, and you need to combine them with domain judgment rather than treating either

as a sufficient statistic.

Multi-Source Relationships

When multiple independent sources assert the same relationship, that's meaningful signal. "Drug A inhibits enzyme B" from one paper is a datum. "Drug A inhibits enzyme B" from five papers, each with different authors, methods, and populations, is a finding. The replication across sources is evidence that the relationship is robust, not an artifact of one study's design or one extraction's error.

Designing your data model to aggregate evidence across sources – rather than storing one relationship record per source – is worth the extra complexity if your corpus is large enough that the same relationship will appear many times. Instead of five edges from document A, B, C, D, and E, you have one edge with a provenance list of five sources. That aggregation enables queries like "how many sources support this relationship?" and "what's the evidence grade for this claim?" It also keeps the graph from exploding in size: the number of unique relationships in a domain is much smaller than the number of relationship mentions across documents.

The aggregation logic needs to handle nuance. Two papers from the same author group might not be independent; you might want to weight them differently than two papers from unrelated labs. A paper that retracts a finding should reduce the evidence count, not leave a stale relationship in place. A paper that asserts the opposite – "drug A does not inhibit enzyme B" – creates a conflict that your model should represent rather than silently merging. These are design choices that depend on your domain and how you plan to use the graph.

Provenance at Query Time

The point of capturing provenance is being able to use it. A graph server that can answer "what's the evidence for this relationship?" rather than just "does this relationship exist?" is a qualitatively different tool. The first supports verification, weighting, and explanation. The second supports only retrieval.

Whether you need that capability depends on your domain and your users. A researcher exploring a graph for hypothesis generation might be satisfied with "drug A is connected to disease B" and not need to drill into sources. A clinician considering a treatment decision needs the evidence. A regulatory submission requires traceability. Design for the most demanding use case you anticipate.

If you do need provenance at query time, design for it from the start. The query interface should support "give me this relationship and its provenance" as a first-class operation. The API response should include source documents, passages, confidence, and whatever else your schema captures. Retrofitting this into a schema that stored relationships without provenance, or into a server that never exposed it, is painful. You'd need to re-ingest to capture what wasn't captured, and you'd need to extend the API to return what wasn't designed to be returned. Get it right early.

Chapter 11: The Identity Server

The identity server is the authoritative component for entity identity across the knowledge graph. It handles the full lifecycle of an entity's identity: resolving a mention to an ID, promoting a provisional entity to canonical status, detecting synonyms, and merging duplicates into a single survivor. All of this must be correct under concurrent access from multiple worker processes and multiple server replicas.

There are a number of authoritative ontologies in the world of medical literature: MeSH for diseases, HGNC for genes, RxNorm for drugs, and others. These are universally known, meticulously maintained, and trusted in that world. The stakes in medicine are high, and the care given to these ontologies reflects that.

Identity Is Load-Bearing

Canonical entities with canonical IDs are the design decision that most separates a useful knowledge graph from a sophisticated extraction exercise. Everything else in the pipeline – extraction, schema design, provenance tracking – is in service of this. Without it, you have a collection of mentions that look like a graph but don't support the reasoning you want. With it, you have a structure where "this drug treats this disease" means the same thing whether it came from a 2010 review article or a 2024 clinical trial, because both references resolve to the same nodes.

The intuition is simple: a knowledge graph is useful when you can ask "what do we know about X?" and get an answer that aggregates

across all sources. That aggregation only works if "X" is the same X everywhere. If one paper calls it "BRCA1," another "breast cancer type 1 susceptibility protein," and a third "the gene encoding the protein that interacts with BARD1," and your graph treats these as three different entities, you've lost the ability to reason. The relationships are there, but they're attached to fragments of an entity rather than the entity itself. Identity resolution is the process of collapsing that multiplicity into something a reasoning system can actually use.

This chapter is about how to do that collapse deliberately, with the right tradeoffs for your domain.

The Scale of the Problem

A concrete illustration from medlit: a single gene across a corpus of thousands of papers might appear as its official symbol (BRCA1), common aliases (breast cancer type 1 susceptibility protein, RNF53), protein product names (BRCA1 protein, breast cancer type 1 susceptibility protein), misspellings (BRCA-1, BRC1), and context-dependent abbreviations (the gene, the protein, the wild-type allele). Authors vary in their conventions. Review articles standardize; primary papers often don't. The same entity, dozens of surface forms.

Multiply that across every entity type in every document. Genes, drugs, diseases, proteins, symptoms, procedures. Each has its own nomenclature, its own abbreviation culture, its own history of naming changes. A drug might appear under its generic name, its brand names in multiple countries, its chemical structure name, a trial identifier, or a lab shorthand. A disease might appear under its ICD code name, a colloquial term, a syndrome eponym, or a molecular subtype designation. Identity resolution isn't a detail you handle in a weekend. It's the central challenge of building a graph that spans diverse sources.

The scale compounds. A corpus of 100,000 documents might produce millions of entity mentions. Many of those mentions will be duplicates in disguise. The deduplication problem – clustering mentions that refer to the same thing – is computationally tractable but requires care. The resolution problem – mapping each cluster

to a canonical identity – is where domain knowledge and authority sources become essential.

Canonical IDs as Primary Keys

The design decision to treat canonical IDs as the primary key rather than derived metadata sounds like an implementation detail. It has deep consequences.

If the primary key is something you derive – a hash of the normalized name, say, or an auto-incrementing integer you assign at ingestion – then identity is an internal concern. Your graph is self-contained. It works, but it doesn't connect to anything else. You can't easily link to external databases that use MeSH terms or HGNC symbols, because your IDs don't match theirs. You can't ingest a new corpus and merge it with an existing one without re-resolving everything. The graph is an island.

If the primary key is a canonical ID from an established authority – a MeSH term for a disease, an HGNC symbol for a gene, an RxNorm code for a drug – then identity is a shared concern. Your graph speaks the same language as the rest of your domain. New documents can be ingested and their entities resolved against the same authorities. External systems can query your graph by the IDs they already use. The graph is interoperable.

Treating canonical IDs as primary keys also forces the right question at the right time. At ingestion, you must answer "what is this entity, really?" You can't defer it to query time, because the graph structure depends on the answer. That forces you to build resolution into the pipeline rather than bolting it on later, when retrofitting is painful.

Authority Lookup

In medicine there are established ontological authorities: MeSH for diseases and symptoms, HGNC for genes, RxNorm for drugs, UniProt for proteins. These are curated, maintained, and widely used. When your extraction produces "ketoconazole," you can look it up in RxNorm and get back a canonical drug ID. When it produces "Cushing's disease," you can look it up in MeSH and get back a term

ID. The authority does the work of disambiguation – "Cushing's disease" (pituitary adenoma) versus "Cushing's syndrome" (the broader clinical picture) – and gives you a stable identifier for each.

Your domain may have equivalents. Legal documents have citation systems; a case or statute can be identified by a standard citation format. Chemistry has InChI and SMILES; a compound can be identified by its structure. Geography has gazetteers and coordinate systems. Library science has ISBNs and OCLC numbers. If your domain has established identifiers, use them. They exist precisely because the domain has an identity problem and the community has invested in solving it.

If your domain doesn't have established authorities, you have choices. You can mint your own IDs – a namespace you control, with IDs you assign to entities as you discover them. This works for self-contained graphs where interoperability isn't a priority. You can use an open identifier scheme – DOI for papers, ORCID for researchers, Wikidata QIDs for general concepts – where it fits. Or you can accept that your graph is self-contained and use internal IDs, with the understanding that merging with other graphs will require a separate resolution step. All of these are legitimate. What matters is making the choice deliberately and documenting it.

The Lookup Chain

When an entity arrives from extraction with a name but no ID, you need a lookup strategy. The naive approach – send every string to the authority API and hope for a match – doesn't scale. Authority APIs have rate limits, latency, and cost. You need a chain: try the cheap, fast options first; escalate to the expensive ones only when necessary.

Exact match first. Normalize the string – lowercase, trim whitespace, expand common abbreviations if you have a mapping – and look it up. Many entities will resolve this way. "Aspirin" in RxNorm, "BRCA1" in HGNC, "type 2 diabetes mellitus" in MeSH. If exact match fails, try fuzzy match. The authority may have "acetylsalicylic acid" when you have "acetyl salicylic acid." Levenshtein distance, or a proper fuzzy matching library tuned to your domain, can find near-misses. Be careful: fuzzy matching can produce false positives. "ACE" might

fuzzy-match to “ache” or “ice” if you’re not constrained to the right vocabulary.

When string-based matching fails, embedding-based reranking can help. Take the entity mention and the candidate matches from the authority, compute embeddings for both, and rank by similarity. This catches semantic matches that string distance misses – “the gene encoding the tumor suppressor” might embed close to “BRCA1” even though the strings share no characters. Embedding search is more expensive and noisier than exact match, so it belongs later in the chain.

The hard part is knowing when to accept a match and when to leave an entity provisional. Too aggressive, and you get false merges: two distinct entities collapsed into one, with relationships from both now incorrectly attached to a single node. Too conservative, and you get missed merges: duplicates in the graph, with the same entity appearing under multiple nodes and your aggregation undercounting the evidence. False merges are generally worse than missed merges – they corrupt the graph in ways that are hard to detect and fix – but the right threshold depends on your domain and what you’re doing with the graph. A graph for hypothesis generation might tolerate more missed merges; a graph for clinical decision support might need to be more conservative about false merges.

Provisional Entities and Promotion

Not everything gets resolved immediately, and that’s fine. A novel compound from a paper published last month might not be in RxNorm yet. A rare disease variant might not have a MeSH term. An institution-specific abbreviation might not map to any authority. These entities still have relationships; they still belong in the graph. They live as provisional entities – nodes with a stable internal ID but no canonical authority ID – until enough evidence accumulates, a better lookup succeeds, or a human reviewer makes a call.

The promotion mechanism is the set of thresholds that govern when a provisional entity earns canonical status. In medlit, the logic is roughly: if a provisional entity appears in enough documents with enough consistency, and if a human has confirmed it or an authority has added it, promote it. The exact thresholds – how many

documents, what consistency, what confirmation – are tunable and domain-specific. Medlit's thresholds reflect LLM extraction characteristics in a medical context: how often the same entity is mentioned across papers, how often extraction produces spurious variants, how often a provisional entity turns out to be a real thing worth promoting versus noise. Your domain may have different dynamics. Design for your domain rather than borrowing wholesale.

Provisional entities are not second-class. They participate in the graph fully. They can have relationships, appear in traversals, and show up in query results. The difference is that they don't have an external canonical ID yet, which means they're not interoperable with systems that expect one. For many use cases, that's acceptable. For others, you'll want a human-in-the-loop process to review high-value provisional entities and either promote them (with a minted ID or a newly available authority match) or merge them into existing canonical entities.

Provenance-Derived Entities and the Citation Graph

Not all entities in the graph come from LLM extraction. Some are derived from document metadata and structure, without touching the model at all. These *provenance-derived* entities are more reliable than extracted ones – they come from authoritative structured fields in the source document rather than from LLM interpretation of prose – and they carry their own class of relationships that are worth treating specially.

In medlit, each ingested paper automatically contributes a set of provenance-derived entities and relationships:

- **Paper** entities (one per ingested document, with its PMC ID as canonical ID)
- **Author** entities (one per credited author)
- **Institution** entities (one per author affiliation)
- `AUTHORED(Author, Paper)` relationships
- `AFFILIATED_WITH(Author, Institution)` relationships
- `DESCRIBED(Paper, entity)` relationships linking each paper to the domain entities most central to it

- `CITES(Paper, Paper)` relationships derived from the paper's reference list

The `CITES` relationships deserve particular attention. Scientific papers include a structured reference list – in JATS XML, the `<ref-list>` element – that names every paper they cite, often with a PMC ID. These citations are parsed directly from the XML and turned into `CITES` edges in the graph, no LLM required. Each cited paper that appears in the reference list becomes a `Paper` entity in the graph, and its outgoing `CITES` edge is asserted with confidence 1.0. This is reliable provenance: the citation relationship is a fact recorded by the authors, not an inference by an extraction model.

The citation graph produced this way is valuable in its own right. It surfaces the intellectual neighborhood of your corpus: which papers cite which, which papers are frequently cited, which are co-cited. Papers that are cited by many papers in your corpus but not yet ingested themselves become natural candidates for corpus expansion – the reference list is a built-in discovery mechanism for related literature.

There is one practical gap: a cited paper that has not been ingested is a stub – a `Paper` entity with a PMC ID as its canonical identifier but only that ID as its name, since no extraction has run on it. Resolving this is straightforward: NCBI's `esummary` API accepts batches of PMC IDs and returns titles, making it possible to populate the `name` field for all stub papers in a single pass of five or so HTTP requests regardless of how many citations are in the corpus. This is much faster than ingesting each paper, and title resolution is all you need for the citation graph to be human-readable.

The identity server handles provenance-derived entities with a short-circuit path: if an entity arrives with an authoritative canonical ID already set (e.g. a PMC ID for a cited paper), the identity server skips the authority lookup and uses that ID directly. This is both faster and more reliable than routing a PMC ID through a name-based lookup chain.

Responsibilities

1. **Canonical ID assignment.** Given a surface-form mention, return a stable entity ID — canonical if an authoritative external source (MeSH, HGNC, RxNorm, etc.) provides one, provisional otherwise. A provisional entity is promotable once domain-defined thresholds are met.
2. **Promotion.** Elevate a provisional entity to canonical status when the domain's promotion policy determines that sufficient evidence has accumulated. Promotion is a one-time, irreversible transition.
3. **Synonym recognition.** Detect when two entities refer to the same real-world concept, using domain-defined criteria such as vector similarity, shared external identifiers, or string normalization.
4. **Merging.** Collapse duplicate entities into a single survivor, redirecting all references from absorbed entities to the survivor. Absorbed entities are retained — not deleted — so that stale external references remain resolvable via a single redirect lookup.

Abstract Interface

```python
from abc import ABC, abstractmethod

class IdentityServer(ABC):

    @abstractmethod
    async def resolve(self, mention: str, context: dict) ->
    str:
        """
        Resolve a mention string to an entity ID.

        Performs domain authority lookup and returns a
canonical ID if one is
        found. Otherwise creates and returns a new
provisional ID. This
        operation must be idempotent: resolving the same
mention twice returns
        the same ID.

        Parameters
        ----------
```

```
        mention:
            The surface form of the entity mention.
        context:
            Domain-defined context (e.g. document ID, domain
name, entity
            type hint, extraction metadata) used by authority
lookup and
            synonym detection.

        Returns
        -------
        str
            A canonical or provisional entity ID.
        """

    @abstractmethod
    async def promote(self, provisional_id: str) -> str:
        """
        Attempt to promote a provisional entity to canonical
status.

        The domain PromotionPolicy determines whether
promotion is warranted.
        Behaviour by current entity status:

        - provisional: checks policy; upgrades if thresholds
are met.
        - canonical: no-op; returns the existing canonical
ID.
        - merged: logs a warning with the stale ID; returns
the survivor's ID.

        This operation must be idempotent.

        Returns
        -------
        str
            The canonical ID (new or pre-existing), or the
survivor ID if
            the entity was merged.
        """

    @abstractmethod
    async def find_synonyms(self, entity_id: str) ->
    list[str]:
```

```
        """
        Return the IDs of entities considered synonymous with
the given entity.

        Synonym criteria are domain-defined and may include
cosine similarity
        above a threshold, shared external identifier, or
string normalisation
        match. This method is read-only; it reports
candidates without merging.

        Returns
        -------
        list[str]
            IDs of synonym candidates, not including
entity_id itself.
            Returns an empty list if no synonyms are found.
        """

    @abstractmethod
    async def merge(self, entity_ids: list[str], survivor_id:
    str) -> str:
        """
        Merge a set of entities into a single survivor.

        All references (relationships, mentions, bundle
edges) pointing to any
        absorbed entity are redirected to the survivor.
Absorbed entities are
        marked status=MERGED with merged_into=survivor_id so
that stale
        external references remain resolvable via a single
lookup.

        This operation must be idempotent: merging
already-merged entities is
        a no-op that returns the survivor ID.

        Parameters
        ----------
        entity_ids:
            The full set of IDs to unify, including the
survivor.
        survivor_id:
```

```
            The ID that will remain after the merge. Must be
a member of
            entity_ids. Determined by the caller via
            DomainSchema.preferred_entity.

        Returns
        -------
        str
            The survivor ID.
        """

    @abstractmethod
    async def on_entity_added(self, entity_id: str, context:
    dict) -> None:
        """
        Event hook called after an entity is inserted or
updated.

        Must be called inside the same transaction as the
entity insert so
        that synonym detection fires only after the row is
durably committed
        and visible. This prevents the race where two
concurrent workers each
        see the other as a merge candidate before either
insert completes.

        Typical implementation:
        1. Embed the entity (if not already embedded).
        2. Call find_synonyms to identify candidates.
        3. If candidates are found, call
DomainSchema.preferred_entity to
           select the survivor.
        4. Call merge for each confirmed synonym pair.

        This event-driven model subsumes batch synonym
sweeps: a batch sweep
        is equivalent to replaying on_entity_added for every
entity in the
        store.
        """
```

Domain-Pluggable Behaviour

The identity server ABC deliberately leaves the following decisions to the domain:

Concern	Where it lives
Authority lookup	Domain implementation (e.g. MeSH API, DBPedia SPARQL, no-op)
Synonym criteria	Domain implementation (e.g. cosine similarity threshold, shared CUI)
Merge survivor selection	`DomainSchema.preferred_entity`
Promotion thresholds	Domain `PromotionPolicy`

Survivor Selection

`DomainSchema` declares an abstract method for survivor selection:

```
@abstractmethod
def preferred_entity(self, candidates: list[BaseEntity]) ->
BaseEntity:
    """
    Given a list of synonym candidates, return the preferred
survivor.

    Domain implementations encode their own preference rules,
e.g.:
    - Prefer canonical status over provisional
    - Prefer the entity with the highest usage_count
    - Prefer the entity with an authoritative canonical_ids
entry
    - Prefer the earlier created_at (more stable,
longer-lived)

    The returned entity must be a member of candidates.
    """
```

`on_entity_added` calls `preferred_entity` to determine `survivor_id` before calling `merge`. Survivor policy is fully domain-controlled and does not belong in the identity server ABC.

Entity Status

Entities carry one of three statuses:

Status	Meaning
`provisional`	Created from a mention with no authoritative external ID. Promotable via the domain promotion policy.
`canonical`	Has a stable external ID (e.g. MeSH term, HGNC symbol) or has been promoted. Promotion is a one-time transition.
`merged`	Absorbed into another entity. Retained for redirect lookups via `merged_into`. Excluded from normal queries.

Merge × Promotion Status Rules

The status of the survivor after a merge:

- **provisional + provisional → provisional.** The merged entity remains promotable via the normal promotion policy.
- **canonical + anything → canonical.** The survivor retains canonical status regardless of the absorbed entity's status.

`promote` behaviour by status:

- **provisional:** check promotion policy, upgrade if warranted.
- **canonical:** no-op; returns the existing canonical ID.
- **merged:** redirect to survivor ID; emit a log warning; do not raise.

Idempotency Contract

Every mutating operation must be safe to call more than once with the same arguments. Workers may retry after transient failures and must not produce inconsistent state by doing so.

Operation	Idempotency mechanism
`resolve`	`INSERT ... ON CONFLICT DO NOTHING` on unique mention/domain key
`promote`	Conditional update `WHERE status = 'provisional'`
`merge`	Advisory lock + existence check; already-merged entities are a no-op
`on_entity_added`	Delegates to `find_synonyms` (read-only) and `merge` (idempotent)

Recommended Implementation: Postgres-Backed

Postgres is the natural backing store because it is already in the stack, provides row-level and advisory locking for cross-replica atomicity, and shares a transaction boundary with entity and relationship writes. No additional coordination service is required for the identity server itself.

Locking Strategy

`resolve` — use `INSERT ... ON CONFLICT DO NOTHING` on a unique index over the normalised mention string and domain. Two concurrent workers resolving the same mention will serialize at the index; the second gets the ID the first created.

`promote` — use `SELECT FOR UPDATE` on the provisional entity row, then check-and-update within the same transaction. This prevents two workers from both evaluating the promotion condition as true and both attempting the transition.

`merge` — acquire a Postgres advisory lock keyed on the lexicographically sorted pair (or set) of entity IDs before beginning the merge transaction. This prevents two workers from merging the same pair in opposite orders, which would produce a deadlock or a double-merge. The advisory lock is released when the transaction commits.

`on_entity_added` — must be called inside the same transaction as the entity insert. Synonym detection (`find_synonyms`) is read-only and acquires no locks. The subsequent `merge` call acquires its own advisory lock as above.

Authority Lookup Caching in `resolve`

Authority lookup is a network call and is not idempotent in the face of transient failures. Results are cached in Redis, shared across all replicas:

1. Check cache keyed on normalised mention + authority source version → hit: use cached ID, skip API call.
2. Miss: call authority API, cache result, then proceed with DB insert.

Negative caching — a "no canonical match" result is also cached with a shorter TTL, so a mention that later acquires an authoritative ID does not remain provisional indefinitely.

Cache key versioning — keys are prefixed with the authority source version, e.g. `resolve:mesh:v2026:{mention}`. A new MeSH release is handled by flushing or re-keying without invalidating unrelated entries.

Residual risk — if the API call succeeds but the process crashes before the DB insert commits, a retry hits the cache, gets the canonical ID, and re-attempts the insert. `ON CONFLICT DO NOTHING` handles this correctly.

Synonym Detection via pgvector

`find_synonyms` uses vector embeddings and cosine similarity via the `pgvector` extension. Approximate nearest-neighbour search at scale is its core capability, and keeping similarity queries inside Postgres means they share the same transaction boundary as entity operations with no additional service.

`on_entity_added` embeds the new entity (if not already embedded) and queries pgvector for neighbours above a domain-defined similarity threshold. The result list is passed to `preferred_entity` and then to `merge`.

Schema Notes

A single `entities` table with a `status` column is preferred over separate tables for provisional and canonical entities. Single-table layout simplifies joins and allows a single index to cover all status-filtered queries.

Merged entities are retained with `status = 'merged'` and a `merged_into` foreign key pointing to the survivor. This supports redirect resolution in a single lookup and preserves provenance for audit and retraction purposes.

Multi-Replica Deployment

The Postgres-backed implementation is correct under any number of concurrent workers or server replicas without additional coordination:

- **`resolve` races** are handled by the unique index and `ON CONFLICT DO NOTHING`. The second worker gets the row the first created; no duplicate entities are produced.
- **`promote` races** are serialized by `SELECT FOR UPDATE`. Only one worker performs the transition; the others see the already-canonical row and return it.
- **`merge` races** are serialized by the advisory lock on the sorted entity ID pair. Two workers attempting to merge the same pair in opposite orders will queue behind the same lock; the second will find the entities already merged and return the survivor ID as a no-op.
- **`on_entity_added` races** — the requirement that this fires inside the entity insert transaction means two workers inserting different entities that are synonyms of each other will each trigger synonym detection after their own insert is visible. The first to detect the other will attempt a merge; the advisory lock ensures this is safe.

Redis is required only for authority lookup caching in `resolve`. If authority lookup is not used (domain is no-op), Redis is not required for the identity server.

Chapter 12: The Ingestion Pipeline

Why Multiple Passes at All

The temptation is to do extraction end-to-end in one shot: send the document to the model, get back entities and relationships, done. That approach fails at scale for reasons that are worth stating explicitly. A single monolithic pass has a single point of failure – if anything goes wrong, you restart from scratch. It produces output that is hard to debug, because you can't inspect intermediate states. And it conflates concerns that are better handled separately: entity extraction, identity resolution, relationship extraction, and assembly are different problems with different failure modes and different recovery strategies.

Staging the pipeline into multiple passes addresses all of this. Each pass has a well-defined input and output. Failures are recoverable: if the extraction pass fails on document 47, you fix the issue and rerun that pass from document 47, not from document 1. Intermediate artifacts are inspectable – you can look at the raw entity extractions before resolution, or the resolved entities before relationship extraction, and see exactly where the pipeline went wrong. The per-document bundle becomes the natural unit of work between passes: each document produces a bundle that can be validated, cached, and merged independently. None of this is medlit-specific. It's good pipeline design for any extraction problem at non-trivial scale.

The medlit batch pipeline uses four stages, each with its own script and a well-defined artifact at its output:

1. **Vocabulary** (`fetch_vocab`): LLM pass over all papers to build a shared vocabulary of canonical entity names and their aliases. Output: `vocab.json` and a seeded synonym cache.
2. **Extract** (`extract`): LLM entity and relationship extraction for each paper, using the vocabulary as context. Output: per-paper `paper_*.json` artifact files in the `extracted/` directory.
3. **Ingest** (`ingest`): Identity-server-based deduplication and canonical ID assignment across all extracted bundles. Output: `entities.json`, `relationships.json`, and an ID map in `merged/`.
4. **Build bundle** (`build_bundle`): Assembles the kgbundle – the loadable artifact consumed by kgserver. Fetches titles for cited papers from NCBI `esummary`. Output: `entities.jsonl`, `relationships.jsonl`, and supporting files in `bundle/`.

That ordering matters. You need a consistent entity vocabulary before extraction can use it. You need resolved entity IDs before you can aggregate relationships across documents. And you need the aggregated merged output before you can build the final bundle. Other orderings are possible, but the principle holds: separate concerns, make each pass debuggable, design for partial failure and restart.

Parsing: Getting to Text

Whatever your source format, you need to get to structured text before you can extract anything. The model reads text; it doesn't read PDF layout or XML tags. JATS XML – the format used by PubMed Central – is medlit's case: a structured representation of journal articles with metadata, abstract, and body sections. Yours might be PDFs, HTML, EPUB, proprietary formats, or plain text that's already clean. The parser's job is to produce a document representation that preserves structure the extractor can use: section boundaries, paragraph boundaries, and the actual text content.

Two decisions matter regardless of format. First, how do you identify section boundaries? In scientific papers, the distinction between Methods, Results, and Discussion carries semantic weight – a claim in Results is different from a claim in Discussion. In legal documents, sections and subsections matter for citation. The parser should expose this structure so downstream passes can use it. Second, how do

you chunk for extraction? Documents are often too long to send to the model in one call. Chunk too small and you lose context – the referent of "it" or "the compound" may be in the previous chunk. Chunk too large and you exceed model context limits, dilute the signal, or hit token budgets that make the run expensive. Overlapping chunks can help: each sentence appears in at least one chunk, so no sentence is orphaned at a boundary. Sentence boundaries are a practical constraint worth respecting – splitting mid-sentence produces fragments that are harder for the model to interpret correctly.

Extraction: The LLM Pass

This is where your schema meets the text. The extraction prompt is not a generic "extract entities and relationships" request. It is a binding of your entity types and relationship types to natural language, written so the model understands exactly what to look for. A good prompt names each entity type with a clear definition, lists each relationship type with its subject and object constraints, and instructs the model to capture provenance alongside content – which passage, what hedging language, what confidence. The prompt is the place where domain expertise gets translated into extraction behavior. A clinician reviewing the prompt should be able to assess whether it matches their understanding of the domain.

The tradeoff between prompt specificity and prompt flexibility is real. A highly specific prompt – "extract only direct assertions, ignore speculative language, require explicit subject-verb-object structure" – tends to produce higher precision and lower recall. The model extracts less, but what it extracts is more reliable. A more flexible prompt – "extract any relationship that might be implied, include hedged claims" – tends to produce higher recall and lower precision. You get more relationships, but more of them will need filtering or correction. There's no universal right answer. The right balance depends on your domain, your tolerance for noise, and what you're doing with the graph. Iteration over the prompt is the design method. You run extraction on a sample, inspect the output, adjust the prompt, repeat. There's no shortcut.

Vocabulary: Building a Shared Terminology

Before extraction, medlit runs a dedicated vocabulary pass over all the papers in the batch. The idea: before you try to resolve "BRCA1," "breast cancer gene 1," and "BRCA1 protein" to the same entity, you establish a shared vocabulary of entity names and their variants. A vocabulary pass asks the LLM a narrower, cheaper question than full extraction – "given the text of this paper, list the distinct named entities you see and their common aliases" – and aggregates the answers across all papers into a canonical name list.

The output of the vocabulary pass feeds directly into extraction. When the extraction prompt runs for each paper, the shared vocabulary is injected as context: "these are the preferred names for entities seen across the corpus; use them." This keeps extraction consistent across workers and across time. Without it, two papers that both mention GPX4 might extract it as "GPX4," "glutathione peroxidase 4," and "phospholipid hydroperoxide glutathione peroxidase" in three different bundles, and identity resolution must sort them out later. With the vocabulary priming the extraction prompt, the model tends to use a consistent preferred form, reducing the deduplication burden downstream.

Not every domain needs a vocabulary pass. If your corpus uses consistent terminology, extraction may produce sufficiently normalized output without it. But medicine, law, and chemistry – any field where the same concept has many names and many names map to the same concept – will see a measurable reduction in deduplication noise. Think of it as schema binding at the lexical level: you're telling the model what things are called before asking it to extract relationships among them.

Deduplication

The same entity extracted from many documents will appear under slightly different names. "Aspirin," "acetylsalicylic acid," "ASA," and "2-acetoxybenzoic acid" are one drug. "Type 2 diabetes," "T2DM," "diabetes mellitus type 2," and "adult-onset diabetes" are one disease. The deduplication stage groups mentions, resolves them to canonical forms, and handles the ambiguous cases. This is

where the gap between "a list of extracted facts" and "a coherent graph" starts to close.

The details vary by domain. In medicine, authority lookup – MeSH, RxNorm, HGNC, and the rest – does much of the work: many apparent synonyms resolve to the same canonical ID automatically. What remains after authority lookup is the residue: novel entities, institution-specific abbreviations, terms that aren't in any vocabulary yet. For those, you need other signals. Embedding similarity can help: mentions that are semantically close in embedding space may be the same entity. So can co-occurrence: if "compound X" and "imatinib" appear in the same document and the context suggests they're the same, that's evidence. The hard cases are the ambiguous ones – "ACE" could be angiotensin-converting enzyme or the gene, "CRF" could be corticotropin-releasing factor or chronic renal failure. Resolving those may require context, domain heuristics, or human review. The universal part: you need a deduplication strategy, and it should run before or alongside relationship extraction so that relationships reference resolved entities, not raw strings.

Assembly

Once you have per-document extractions – entities resolved, relationships extracted – you need to merge them into a coherent whole. Assembly is not just concatenation. When multiple documents assert the same relationship, that's meaningful signal. "Drug A treats Disease B" from one paper is weaker than "Drug A treats Disease B" from five independent papers. The assembly stage should aggregate evidence across sources: one relationship record with a provenance list, not five duplicate edges. That aggregation is what makes the graph useful for reasoning – you can weight relationships by how many sources support them, filter by evidence type, and detect when sources conflict.

The structure of the final bundle is worth thinking about carefully before you start. What does a "document" in your graph look like? Is it a node with metadata and outgoing edges to the relationships it supports? Are relationships first-class with document references, or are documents first-class with relationship references? The choice affects query patterns, provenance traversal, and how you handle

updates when you re-ingest a document with corrections. Changing the bundle structure later, once you have data and downstream consumers, is expensive. Get it right early.

Progress Tracking and Resumability

Large ingestion runs fail partway through. A run over 100,000 documents will hit rate limits, network timeouts, model outages, or your own mistakes. If the pipeline has no notion of progress, you restart from zero every time. That's acceptable for a research prototype. It's not acceptable for something you run regularly.

Design for restartability from the beginning. Each document should have a processing status: not started, in progress, completed, failed. The pipeline should record which documents have been fully processed and which haven't. On restart, it should skip completed documents and resume from the first incomplete one. Checkpointing within a document – if a single document requires multiple LLM calls, record which chunks have been processed – can help for very long documents, though the document is usually the right granularity. The progress store should be persistent and survive process restarts. This isn't glamorous work. It's the difference between a pipeline you can run once as a demo and a pipeline you can run every week as part of your workflow.

Design Principles

The concepts above translate directly into four implementation commitments.

Dedup-on-write. The `IdentityServer` resolves entity identity and detects synonyms incrementally as each entity is written. There is no global deduplication pass over the corpus; papers can be ingested concurrently and the identity server handles merging correctly under concurrent access.

Per-paper atomicity. Each paper moves through stages independently. A failure at any stage leaves the paper at its last committed status; the next available worker picks it up and retries. No paper's failure affects any other.

Durable checkpoints. Raw fetched text and raw LLM extraction output are stored durably before any graph writes. This means a schema change, extraction bug, or infrastructure failure can be recovered from without re-fetching or re-paying LLM costs.

Shared pipeline code. The MCP tool and the batch runner call the same stage functions. There is no separate implementation for interactive versus batch use.

Stages

1. Fetch

Retrieves raw text or XML for a PMC ID from the PMC API and stores it in `ingest_jobs.raw_text`. This stage is pure I/O and completely independent of all other work. A previously fetched paper never needs to be re-fetched.

2. Extract

Runs the LLM extraction call for a paper. Before calling the LLM, the worker queries the live graph for the most frequent entities of each relevant type and injects them as context. This keeps extraction consistent across workers and across time — the context improves as the corpus grows.

The raw LLM output is stored in `ingest_jobs.raw_extraction` and also written to a paper artifact file (see below) before any graph writes occur.

3. Ingest

Writes entities and relationships from `raw_extraction` to the graph. For each entity:

1. `identity_server.resolve(mention, context)` — returns a canonical or provisional entity ID, creating one if needed, idempotently.
2. `identity_server.on_entity_added(entity_id, context)` — triggers synonym detection and merge.

Both calls happen **inside the same transaction as the entity**

insert, so synonym detection fires only after the row is durably committed and visible. This prevents the race where two concurrent workers see each other as merge candidates before either insert completes.

Relationship writes are in the same transaction as the entity inserts for the paper. There is no gap between them.

Work Queue

A Postgres table coordinates all work:

```
CREATE TABLE ingest_jobs (
    pmcid          TEXT PRIMARY KEY,
    status         TEXT NOT NULL DEFAULT 'pending',
                   -- pending → fetching → fetched
                   --        → extracting → extracted
                   --        → ingesting → done | failed
    raw_text       TEXT,
    raw_extraction JSONB,
    error          TEXT,
    attempts       INT DEFAULT 0,
    updated_at     TIMESTAMPTZ DEFAULT now()
);
```

Workers claim jobs with:

```
SELECT * FROM ingest_jobs
WHERE status = 'pending'
FOR UPDATE SKIP LOCKED
LIMIT 1;
```

`SKIP LOCKED` provides a distributed work queue with no additional infrastructure: no races, crash-safe, and naturally load-balanced across any number of workers or server instances. A crashed worker releases its lock when its connection drops and the job is retried by the next available worker. The `attempts` counter enables dead-lettering after N failures, leaving the job at `status='failed'` with the error recorded.

Progress is trivially observable:

```
SELECT status, count(*) FROM ingest_jobs GROUP BY status;
```

Paper Artifact Files

After `extract_stage` stores `raw_extraction` in Postgres, it also writes a per-paper artifact file using atomic write-then-rename:

```
tmp = artifact_dir / f".tmp_{pmcid}_{os.getpid()}.json"
tmp.write_text(json.dumps({
    "pmcid": pmcid,
    "raw_text": raw_text,
    "raw_extraction": raw_extraction,
}, indent=2))
tmp.rename(artifact_dir / f"paper_{pmcid}.json")
```

Including both `raw_text` and `raw_extraction` makes each file a self-contained record of everything that happened for that paper.

These files serve three purposes:

Recovery. If Postgres is lost, the artifact directory is sufficient to repopulate the graph. For extraction-side schema changes (new entity types, revised prompts), reset to `status='fetched'` and re-run from `raw_text`. For ingest-side changes only, reset to `status='extracted'` and re-run from `raw_extraction`. In both cases the identity server runs fresh and rebuilds all merges correctly.

Auditability. A human-readable record of what was fetched and what the LLM extracted, independent of any subsequent graph operations.

Retraction support. When a paper is retracted, its artifact file provides an exact record of every claim it contributed to the graph. Provenance-aware graph queries can then down-weight or exclude retracted sources. The artifact is the supporting record for that process; a `retracted` flag on `ingest_jobs` is the mechanism.

The artifact directory can be a local volume, NFS mount, or S3-compatible store.

Repopulation from artifacts:

```
python -m kgserver.ingest --from-artifacts ./artifacts/
--ingest-workers 16
```

Reads `paper_*.json` from the directory, inserts each into `ingest_jobs` with the appropriate status and fields populated from the file (`INSERT ... ON CONFLICT DO NOTHING` skips papers

already in the table), then lets the normal worker pool handle the rest.

Parallelism

Different stages have different bottlenecks and should have independently tunable worker counts:

Stage	Bottleneck
Fetch	PMC API rate limits
Extract	LLM token budget / cost
Ingest	Postgres / identity server locks

The batch runner accepts separate concurrency limits per stage. Running multiple instances of the batch runner across machines is safe; all instances share the same `ingest_jobs` table and coordinate via `SKIP LOCKED`.

Batch Ingestion

The batch pipeline is driven by a shell script that sequences the four stages in order:

```
# Run all four stages over a named list of papers
./run-ingest-new.sh --list ferroptosis      # two-paper test
set
./run-ingest-new.sh --list smorgasbord     # 39-paper corpus
```

Each stage is a Python module invoked with `uv run python -m`:

```
# Stage 1: vocabulary
uv run python -m examples.medlit.scripts.fetch_vocab \
    --input-dir pmc_xmls --output-dir vocab --papers
    PMC12345.xml,...

# Stage 2: extraction
uv run python -m examples.medlit.scripts.extract \
    --input-dir pmc_xmls --output-dir extracted \
    --vocab-file vocab/vocab.json --papers PMC12345.xml,...

# Stage 3: ingest (identity-server deduplication)
```

```
uv run python -m examples.medlit.scripts.ingest \
    --bundle-dir extracted --output-dir merged
    --use-identity-server

# Stage 4: build bundle
uv run python -m examples.medlit.scripts.build_bundle \
    --merged-dir merged --bundles-dir extracted \
    --output-dir bundle --pmc-xmls-dir pmc_xmls
```

Stages 1 and 2 are embarrassingly parallel at the paper level and can be run with multiple workers. Stage 3 parallelizes authority lookups (MeSH, UniProt, HGNC) within a single run using `asyncio.gather` with a semaphore, so HTTP calls to authority APIs happen concurrently rather than sequentially. Stage 4 is fast and single-threaded – its main network cost is the batched NCBI `esummary` call for cited-paper titles, which takes only a handful of requests regardless of corpus size.

MCP Tool

The MCP tool is a convenience for a user who wants to pull in one or a few papers during a query session and have them available immediately. It is not intended for bulk operations.

The tool calls the same underlying pipeline stages as the batch CLI – fetch, extract, ingest, build bundle – but runs them synchronously in a background thread so the MCP server stays responsive. If the paper has already been ingested, it returns immediately with a status message. For large lists of papers, use the batch CLI instead; the MCP tool is optimized for single-paper interactive use.

```
@mcp.tool()
async def ingest_paper(pmcid: str) -> dict:
    """
    Fetch, extract, and ingest a single PMC paper into the
knowledge graph.
    Runs the full pipeline (fetch → extract → ingest →
build_bundle) and
    makes the paper available for querying on return.
    """
    # Check if already ingested
    existing = check_ingest_status(pmcid)
    if existing == "done":
```

```
        return {"status": "already_ingested", "pmcid": pmcid}

    # Run all pipeline stages via the same function used by
    the batch worker
    await asyncio.get_event_loop().run_in_executor(
        None, _run_pass2_pass3_load, workspace, bundles_dir,
        merged_dir, output_dir
    )
    return {"status": "ingested", "pmcid": pmcid}
```

The `_run_pass2_pass3_load` function is the shared implementation used by both the MCP tool and the background ingest worker. It runs `ingest`, `build_bundle`, and `load_bundle_incremental` in sequence, so that after it returns the paper is live in the graph without a server restart.

Extraction Output Format

The artifact file captures what the LLM decided, before the identity server assigns any entity IDs. Mentions and evidence strings include their location in the source document so that any claim can be verified against the original text.

`raw_text` is stored as the original PMC XML rather than stripped plain text. PMC XML has explicit section labels (`<sec>`, `<title>`, `<p>`) that make section and paragraph extraction reliable, and preserving the structure means location references remain valid if the artifact is re-ingested later.

Most paper metadata and cited references are available as structured fields in the PMC XML and are parsed directly by the fetch stage rather than extracted by the LLM. This makes them reliable and cheap — no prompt engineering required. Each cited PMC ID is also a candidate for further ingestion, making the reference list a natural source for corpus expansion.

```
{
  "pmcid": "PMC12345",
  "extracted_at": "2026-03-17T14:23:00Z",
  "model": "claude-sonnet-4-6",
  "metadata": {
    "title": "Serum cortisol as a diagnostic marker for
    hypercortisolism",
```

```
    "authors": [
      { "name": "Jane A. Smith",  "institution":
      "Massachusetts General Hospital" },
      { "name": "Robert T. Chen", "institution": "Harvard
      Medical School" }
    ],
    "publication_date": "2024-09-15",
    "journal": {
      "name": "Journal of Clinical Endocrinology &
      Metabolism",
      "issn": "0021-972X",
      "volume": "109",
      "issue": "4",
      "pages": "1123-1131"
    }
  },
  "references": [
    {
      "pmcid": "PMC98765",
      "doi": "10.1210/clinem/dgad001",
      "authors": ["Johnson B", "Lee K"],
      "title": "Urinary free cortisol in Cushing syndrome
      diagnosis",
      "journal": "Endocrine Reviews",
      "year": "2023"
    }
  ],
  "entities": [
    {
      "mention": "cortisol",
      "type": "Biomarker",
      "locations": [
        { "section": "Abstract", "paragraph": 1, "sentence":
        2 },
        { "section": "Results",  "paragraph": 2, "sentence":
        1 },
        { "section": "Results",  "paragraph": 4, "sentence":
        3 }
      ],
      "attributes": {
        "value": "elevated",
        "specimen": "serum"
      }
    }
  ],
```

```
  "relationships": [
    {
      "subject_mention": "cortisol",
      "predicate": "indicates",
      "object_mention": "hypercortisolism",
      "evidence": "Serum cortisol levels were elevated in all
      patients diagnosed with hypercortisolism.",
      "evidence_locations": [
        { "section": "Results", "paragraph": 2, "sentence": 1
        }
      ]
    }
  ]
}
```

Key properties:

- **Mentions, not IDs.** The artifact records what the LLM said. Entity IDs are assigned by `identity_server.resolve()` during ingest and are not present here.
- **All locations for each entity.** The same entity may be mentioned many times across a paper. Recording all locations supports `usage_count` computation and provides the full evidentiary basis for the entity.
- **Evidence strings with location.** The verbatim text supporting each relationship, with its position in the document. `evidence_locations` is a list to handle cases where the supporting evidence spans multiple sentences or where the antecedent of a pronoun appears in a prior sentence.
- **Model and timestamp.** Records exactly what produced this output, which is essential when comparing extractions before and after a prompt or model change.
- **No graph state.** Nothing about merges, status, or canonical IDs. That is all rebuilt fresh by the identity server on each ingest.

Shared Pipeline Code

The four pipeline stages share their core logic across both the batch CLI and the MCP/server path. The batch CLI calls stage scripts directly; the MCP tool and background ingest worker call `_run_pass2_pass3_load`, which sequences the ingest, build_bundle,

and load_bundle_incremental steps using the same underlying functions:

```
def _run_pass2_pass3_load(
    workspace_root: Path,
    bundles_dir: Path,
    merged_dir: Path,
    output_dir: Path,
) -> None:
    """Run ingest, build_bundle, and load_bundle_incremental.
    Raises on failure."""
    run_ingest(bundle_dir=bundles_dir, output_dir=merged_dir,
...)
    run_build_bundle(merged_dir, bundles_dir, output_dir)
    load_storage.load_bundle_incremental(manifest,
str(output_dir))
```

`run_ingest` is the identity-server deduplication stage. `run_build_bundle` assembles the kgbundle including NCBI title fetching. `load_bundle_incremen` pushes the new bundle into the live graph storage without a restart.

The vocabulary and extraction stages are not in this shared path – they are CLI-only for batch runs, since interactive single-paper ingestion via the MCP tool skips the vocabulary pass (the vocabulary built from the existing corpus is already embedded in the seeded synonym cache). For the MCP use case, the paper is extracted with the current vocabulary as context, then ingested, and the bundle is rebuilt and reloaded.

Part IV: What It Makes Possible

Chapter 13: What Your Graph Can Do

The value of the graph is in what grounded reasoning becomes possible, not in the serving layer. Evidence, not instruction.

The Server Is Not the Point

This chapter is about what becomes possible once your graph exists, not about how to build a particular server. The medlit project has a serving layer – kgserver – with REST, GraphQL, MCP, and persistent storage backends. You might build something like it, or you might expose your graph through a completely different interface, or you might not serve it externally at all. The infrastructure choices are yours. What this chapter is really about is the capability space: what can a well-constructed knowledge graph actually do for someone?

That distinction matters because it's easy to conflate "I have a graph" with "I have a graph server." The graph is the data structure and the relationships it encodes. The server is one way to expose it. You could serve the same graph through a REST API, a GraphQL endpoint, an MCP server, a SPARQL endpoint, a custom query language, or no server at all – just load it into memory and run Python scripts against it. The capabilities we're about to describe – direct querying, visualization, grounding LLMs, hypothesis generation – are capabilities of the graph. The server is a delivery mechanism. Choose one that fits your users and your deployment constraints; don't let the choice of server obscure what the graph itself enables.

Direct Querying

The basics: entity lookup, relationship queries, graph neighborhood traversal. These are useful and often sufficient for many applications. The interesting design question isn't which API style (REST, GraphQL, something else) but what the right query primitives are for your domain. What questions will your users actually ask?

A biomedical researcher might ask: "What drugs are known to treat this condition?" "What genes are associated with this disease?" "What's the evidence for this drug-gene interaction?" A legal researcher might ask: "What cases cite this statute?" "What statutes does this case interpret?" The primitives that support these questions – get entity by ID, get relationships of type X from entity Y, get N-hop neighborhood, filter by provenance – are similar across domains. The *semantics* of what counts as a good answer differ. A drug-disease "treats" relationship in medicine has different evidentiary standards than a case-statute "cites" relationship in law. Your query interface should expose primitives that map cleanly to your domain's question types, not force users to translate their questions into a generic graph query language.

At minimum, you need: entity lookup (by name or canonical ID), relationship enumeration (what connects to this entity, and how), and some form of traversal (neighbors, N-hop expansion, path finding). Provenance-aware queries – "give me this relationship and its sources" – belong in the primitive set if your domain cares about evidence, which most serious domains do. Everything else is optimization.

Graph Visualization

Chapter 9 covered visualization as a diagnostic tool for pipeline development. Here the question is different: what does visualization do for an end user exploring the graph?

A browsable, zoomable view of entities and relationships lets users navigate structure that would be tedious to reconstruct from query output. "Show me everything connected to this drug" produces a list; a force-directed layout produces a picture where clusters, bridges, and outliers are visible at a glance. For exploration and discovery –

"what's in this neighborhood?" "what connects these two things?" – visualization often beats tabular output. The implementation cost is modest if you already have the query primitives; the value for users who think spatially about their domain is high. If you build nothing else on top of your graph, build this.

Grounding LLM Inference

The pattern that changes what a language model can do: instead of asking a model to reason from its training data, give it structured, typed, provenance-tracked claims from your graph and ask it to reason from those. The difference in reliability is substantial. A model hallucinating over raw text and a model reasoning over a curated graph with explicit provenance are doing qualitatively different things, even if they look similar from the outside. This is the integration that makes a knowledge graph more than a database.

The mechanics are straightforward. A user asks a question. Your system retrieves relevant subgraphs – entities and relationships that match the question's scope – and injects them into the model's context. The model reasons over that context and produces an answer. The answer is grounded in the retrieved graph, not in the model's training. You can cite the sources. You can trace the reasoning path. When the graph is wrong, you fix the graph; you don't retrain the model.

The retrieval step matters. "Relevant" means different things for different questions. A question about a specific drug might need that drug's neighborhood, its indications, its interactions, and the evidence for each. A question about a disease might need the disease's subtypes, associated genes, known treatments, and the studies that support those links. Designing the retrieval logic – what subgraph to fetch for what question – is where domain knowledge enters. A generic "fetch entities mentioned in the question" often works; a retrieval strategy tuned to your schema and your users' question patterns works better. The graph gives you something to retrieve. The retrieval strategy determines how well the model uses it.

MCP as the Integration Point

The Model Context Protocol is worth understanding as an architectural pattern, not just as a specific technology. The idea is that a knowledge graph should be a first-class context source for LLM-based systems – something that agents, assistants, and reasoning pipelines can query as naturally as a human researcher would reach for a reference database. Whether you use MCP specifically or some other integration approach, the principle is sound: your graph is most powerful when it's actively grounding inference, not sitting passively waiting to be queried by humans.

MCP defines a standard way for AI systems to discover and call tools. A knowledge graph exposed as an MCP server offers tools like "find entities matching this query," "get the neighborhood of this entity," "retrieve relationships of type X." An LLM-powered assistant with access to that server can answer domain questions by querying the graph, synthesizing the results, and citing the sources. The user gets an answer grounded in your curated knowledge rather than in the model's training distribution. The integration is loose: the model doesn't need to know your schema in advance; it discovers the available tools and uses them. That looseness is a feature. Your graph can evolve without breaking every consumer.

If you're not using MCP, the same pattern applies. The graph needs to be queryable by whatever system is doing the reasoning. REST, GraphQL, or a custom API all work. The architectural point is that the graph should be *available* to the reasoning layer, not a separate system that humans query manually. Passive retrieval – human runs query, copies result, pastes into chat – is a fallback. Active grounding – the reasoning system queries the graph as part of generating its answer – is the target.

BFS Queries

This section is unusually detailed. I hope it will be helpful enough to justify that. Here I describe a JSON-based query language that I designed for my app. The guiding design principles were LLM friendliness, both in the use of JSON syntax and in the conservation of context window budget. Initially I favored GraphQL but later

learned that LLMs really do prefer JSON. And context window is a precious commodity in LLM work.

The BFS query (breadth-first search) provides a single, general-purpose mechanism for retrieving subgraphs from the knowledge graph. It is designed to be constructed easily by an LLM while minimizing unnecessary context window consumption in the response.

The design separates two orthogonal concerns:

- **Topology**: BFS from one or more seed nodes up to a specified hop depth defines which nodes and edges are included in the subgraph. Filtering has no effect on this.
- **Presentation**: Node and edge filters determine which items in that subgraph receive full metadata vs. a minimal stub. This is purely a serialization decision applied after the subgraph is computed.

Stub items carry only enough information to understand the graph's topology without consuming context on irrelevant details. The LLM sees the full shape of the neighborhood while receiving rich data only where it matters.

Query Format

```
{
  "seeds": ["<entity_id>", ...],
  "max_hops": <int>,
  "node_filter": {
    "entity_types": ["<type>", ...]
  },
  "edge_filter": {
    "predicates": ["<predicate>", ...]
  }
}
```

Field	Required	Description
`seeds`	Yes	Array of one or more canonical entity IDs to use as BFS starting points. All seeds are expanded simultaneously; the result is the union of their neighborhoods.
`max_hops`	Yes	Maximum graph distance from any seed node. Values of 1–3 are typical; larger values may return very large subgraphs.
`node_filter`	No	Controls which nodes receive full metadata. Nodes not matching the filter appear as stubs. Omit to receive full data on all nodes.
`node_filter.entity_types`	Yes	List of entity type names. A node matches if its type is in this list.
`edge_filter`	No	Controls which edges receive full metadata including provenance. Edges not matching appear as stubs. Omit to receive full data on all edges.

Field	Required	Description
`edge_filter.predicates`	No	List of predicate names. An edge matches if its predicate is in this list.

- If `node_filter` is omitted entirely, all nodes in the subgraph receive full data.
- If `edge_filter` is omitted entirely, all edges receive full data including provenance.
- Omitting both filters is appropriate for small subgraphs or debugging, but will produce large responses on dense neighborhoods.
- Multiple seeds are useful when you want to explore the shared neighborhood of several entities simultaneously – for example, finding publications co-authored by two researchers, or finding diseases connected to a set of genes.
- If you do not yet have a canonical entity ID, call `search_entities()` first to resolve a name to an ID.

Response Format

```
{
  "seeds": ["<entity_id>", ...],
  "max_hops": 2,
  "node_count": <int>,
  "edge_count": <int>,
  "nodes": [
    {
      "id": "<entity_id>",
      "entity_type": "<type>",
      "<additional fields>": "..."
    }
  ],
  "edges": [
    {
      "subject": "<entity_id>",
      "predicate": "<predicate>",
      "object": "<entity_id>",
      "<additional fields>": "..."
    }
```

```
  ]
}
```

Full Node

A node that matches the `node_filter` (or when no filter is specified) includes all available metadata:

```
{
  "id": "PUB:PMC2386281",
  "entity_type": "Publication",
  "title": "The Diagnosis of Cushing's Syndrome",
  "canonical_id": "PMID:18493314",
  "year": 2008,
  "journal": "Reviews in Endocrine and Metabolic Disorders",
  "authors": ["Stewart PM"],
  "abstract_snippet": "..."
}
```

Stub Node

A node that does not match the `node_filter` appears as a stub with only identity information:

```
{
  "id": "PERSON:67890",
  "entity_type": "Person"
}
```

Full Edge

An edge that matches the `edge_filter` (or when no filter is specified) includes all provenance and metadata:

```
{
  "subject": "PERSON:12345",
  "predicate": "AUTHORED",
  "object": "PUB:PMC2386281",
  "confidence": 0.97,
  "provenance": [
    {
      "source_doc": "PMC2386281",
      "section": "metadata",
      "method": "structured_extraction",
      "evidence_type": "primary_authorship"
    }
  ]
```

```
}
```

Stub Edge

An edge that does not match the `edge_filter` appears with topology only:

```
{
  "subject": "PERSON:12345",
  "predicate": "COLLEAGUE_OF",
  "object": "PERSON:67890"
}
```

LLM Prompt

The following section can be included in a system prompt or tool description to instruct an LLM how to construct BFS queries.

To explore the knowledge graph, call `bfs_query()` with a JSON body. The query performs a breadth-first search from one or more seed nodes and returns the resulting subgraph.

Nodes and edges in the subgraph are either **full** (all metadata and provenance included) or **stub** (identity only), depending on the filters you specify. Filtering affects only what data is returned, not which nodes and edges are included in the subgraph.

If you do not yet have a canonical entity ID, call `search_entities()` first.

Query structure

```
{
  "seeds": ["<id>", ...],        // one or more starting
  entity IDs
  "max_hops": <int>,             // graph distance from seeds
  (1-3 recommended)
  "node_filter": {               // optional: which nodes get
  full data
    "entity_types": [...]        // e.g. ["Publication",
    "Disease", "Drug"]
  },
  "edge_filter": {               // optional: which edges get
  full data + provenance
    "predicates": [...]          // e.g. ["AUTHORED",
    "TREATS", "INHIBITS"]
```

```
  }
}
```

Stub nodes contain only `{id, entity_type}`. Stub edges contain only `{subject, predicate, object}`. Omitting a filter entirely returns full data for all nodes or edges respectively.

Example 1: Find an author's publications with provenance

You know Dr. Stewart's entity ID and want to retrieve her publications along with the evidence that links her to each one. You don't need full metadata on other entities in the neighborhood.

```
{
  "seeds": ["PERSON:12345"],
  "max_hops": 1,
  "node_filter": {
    "entity_types": ["Publication"]
  },
  "edge_filter": {
    "predicates": ["AUTHORED"]
  }
}
```

Result: Full data on Publication nodes, full provenance on AUTHORED edges. Any other nodes or edges at hop 1 (e.g. institutional affiliations) appear as stubs.

Example 2: Explore a disease neighborhood, focusing on drugs

You want to understand what drugs are connected to Cushing's syndrome within two hops, without being overwhelmed by the full metadata of every gene, symptom, and pathway in the neighborhood.

```
{
  "seeds": ["MeSH:D003480"],
  "max_hops": 2,
  "node_filter": {
    "entity_types": ["Drug"]
  }
}
```

Result: Full data on Drug nodes. All other node types (Disease, Gene, Symptom, etc.) appear as stubs. All edges appear as stubs since no `edge_filter` targets specific predicates – you can see the

topology of the neighborhood without consuming context on provenance you haven't asked for.

Example 3: Find shared connections between two entities

You want to explore what two researchers have in common – shared publications, shared diseases they've written about, or shared collaborators – within two hops of either of them.

```
{
  "seeds": ["PERSON:12345", "PERSON:67890"],
  "max_hops": 2,
  "node_filter": {
    "entity_types": ["Publication", "Disease"]
  },
  "edge_filter": {
    "predicates": ["AUTHORED", "DISCUSSES"]
  }
}
```

Result: BFS expands from both researchers simultaneously. The returned subgraph is the union of both neighborhoods. Full data on Publications and Diseases they connect to; full provenance on AUTHORED and DISCUSSES edges. Everything else is stubbed. Nodes reachable from both seeds appear once, making shared connections directly visible.

Design Considerations

Topology and presentation are orthogonal

The subgraph returned by a BFS query is determined entirely by `seeds` and `max_hops`. Filters have no effect on which nodes or edges are included – they only control how much data each item carries in the response. This means the LLM always sees an accurate picture of the graph's topology, regardless of what it filtered for. A stub node is not a missing node; it is a node whose full metadata was not requested.

Why stubs rather than omission

Omitting non-matching nodes entirely would produce a misleading picture of the graph. If Dr. Jones appears as a stub rather than disappearing, the LLM knows that Stewart has a connection to another person in the graph, and can issue a follow-up query for Jones

if needed. Omission would make the graph appear sparser than it is, causing the LLM to miss connections it doesn't know to ask about.

Edge provenance is expensive context

A single edge with strong multi-source support may carry a long provenance list – source documents, confidence scores, evidence types, extraction methods. Returning full provenance on every edge in a two-hop neighborhood would dominate the context window on most queries. The `edge_filter` gives the LLM precise control over where that cost is paid.

Multiple seeds enable relational queries

Accepting an array of seeds rather than a single seed allows the LLM to express relational questions – "what do these entities have in common?" – without requiring a specialized query type. The union semantics are simple to implement and simple to reason about.

Filters are independently optional

`node_filter` and `edge_filter` compose independently. You can request full node data with stub edges (useful when you want entity details but not provenance), full edge provenance with stub nodes (useful when you want to audit support for a relationship without loading entity metadata), both (for focused high-detail queries), or neither (for small graphs or debugging). No special cases are required.

BFS depth guidance

Depth 1 is appropriate for direct relationships: an author's publications, a drug's known indications, a gene's associated diseases. Depth 2 surfaces indirect connections: diseases associated with genes that a drug targets, co-authors of papers a researcher has written. Depth 3 and beyond can return very large subgraphs on well-connected nodes and should be used with targeted filters. When in doubt, start at depth 1 and increase.

Hypothesis Generation

The capability that arguably justifies building the graph in the first place, and the one that most clearly connects back to Chapter 2's

Robot Scientist. Graph traversal as a discovery tool: not "what do we know about X" but "what's adjacent to X that hasn't been studied," "what entities are structurally similar to X in the graph," "what relationships exist between X and Y that no single paper asserts but that follow from combining multiple sources." These are queries that are impossible over raw text and natural over a well-constructed graph.

Consider a concrete example. Drug A treats disease D. Gene G is associated with disease D. Drug B modulates gene G. No single paper may state that drug B is worth testing for disease D. The inference follows from combining three relationships that exist in the graph. A researcher who had read all the relevant papers might make that connection; the graph makes it queryable. "What drugs modulate genes associated with this disease?" is a traversal. The results are candidate hypotheses: drug-disease pairs that the graph implies but that may not have been studied together. Some will be known, some will be nonsense, some will be novel and worth investigating. The graph doesn't decide which are which. It surfaces candidates that a human or a downstream system can filter and prioritize.

Structural similarity is another pattern. Two entities are structurally similar if their neighborhoods in the graph look alike – similar relationship types, similar connected entity types, similar topology. If drug X is structurally similar to drug Y, and drug Y treats condition Z, then drug X might be worth testing for Z. The graph encodes the structure; the similarity query exploits it. Again, the result is a candidate set, not a conclusion. The value is in narrowing the space of possibilities to something a researcher can evaluate.

Should Hypothesis Generation Be Baked In?

A question worth sitting with rather than answering too quickly. There's an argument that hypothesis generation is generic enough – graph traversal, structural similarity, gap detection – that it belongs in the framework rather than in domain-specific code. There's a counterargument that what counts as an interesting hypothesis is deeply domain-specific, and that baking it in prematurely produces a mechanism that nobody actually wants. The honest answer is probably that the *primitives* (traversal, similarity, subgraph queries)

belong in the framework, and the *interpretation* of what makes a result interesting belongs in domain code or in the prompts given to whatever LLM is reasoning over the results.

The framework should provide: neighborhood queries, path finding, structural similarity metrics, and the ability to express "entities connected by relationship type R" and "entities that share property P." The framework should not provide: "generate interesting hypotheses for drug discovery" or "suggest research directions." Those require domain-specific filters (what counts as a plausible drug target?), domain-specific prioritization (what's already been studied?), and domain-specific presentation (how do you explain the suggestion to a researcher?). The line between generic and domain-specific is fuzzy, but the principle holds: build reusable primitives; let domain code compose them into domain-specific capabilities.

Returning to the Dream

This section returns to the themes of Part I. Chapter 2 presented the long-standing vision of machines that reason over explicit knowledge – the dream that motivated semantic networks, expert systems, Cyc, and the Semantic Web. Chapter 4 argued that representation is the prerequisite for principled reasoning: that grounded inference requires an explicit, inspectable structure, not just statistical fluency. The chapters that followed have laid out the engineering: extraction pipelines, schema design, authority lookup, and the integration of a knowledge graph with a language model for synthesis and explanation. A functioning system built along those lines is not the Robot Scientist that Adam and Eve embodied – it does not design and execute experiments in a lab – but it is the closest thing currently buildable. The gap between what the preceding chapters have shown how to construct and what those systems demonstrated is narrower than it looks. The architecture is the same: represent the domain explicitly, generate candidates from structure, ground the reasoning in that representation. The substrate has changed from formal logic and lab robotics to graphs and language models. The need for that explicit layer has not.

Adam and Eve had curated knowledge bases, hypothesis generation from structural patterns, and automated experiment design. You

have extracted knowledge from literature, hypothesis generation from graph traversal, and the ability to ground an LLM in that graph for synthesis and explanation. The substrate is different – they used formal logic and lab robotics; you use graphs and language models – but the architecture is recognizably the same. Represent the domain explicitly. Generate candidates from structure. Ground the reasoning in that representation. The tools have changed. The need for that layer has not.

Chapter 14: The Augmented Researcher

What Machines Would See That We Can't

A human researcher in a specialized field might read a few thousand papers over a career. A knowledge graph built from a comprehensive corpus has ingested millions. That scale difference is not just quantitative; it changes what kinds of patterns become visible.

Consider confirmation bias. A researcher with a hypothesis tends to notice evidence that supports it and to underweight evidence that doesn't. This isn't a character flaw; it's how attention works. When you're reading papers one at a time, your prior beliefs shape what you notice, what you remember, and what you connect. A graph doesn't have prior beliefs. It encodes what the literature asserts, and a traversal query doesn't care whether the result confirms or contradicts your favorite theory. The graph surfaces connections that a human reader, biased toward coherence with existing beliefs, might have skimmed past. That doesn't make the graph right and the human wrong. It makes them different. The graph offers a view that isn't filtered through a single researcher's expectations.

Prestige bias works similarly. A finding from a famous lab or a high-impact journal gets more attention than the same finding from an unknown group or a niche venue. Citation networks amplify this: papers that are already well-cited get cited more, in a feedback loop that the Matthew effect describes. A knowledge graph built from a broad corpus can include relationships from papers that nobody cites. The graph doesn't know which papers are prestigious. It knows which

relationships were extracted. A query over the graph can surface a connection that appeared in an obscure regional journal twenty years ago and was never picked up by the mainstream literature. Again, that doesn't make the obscure paper right. It makes it *visible* in a way that citation-based discovery systematically hides it.

Recency bias is the flip side. Newer work gets more attention than older work, partly because it's easier to find and partly because the field has collectively decided that recent results matter more. But important findings sometimes sit in the literature for decades before someone connects them to a new context. A graph that spans the full temporal range of a corpus can surface those connections. "What did we know about X in 1990?" is a query that citation networks handle poorly – they tend to show you what's cited now, which skews recent – but a graph can answer it directly.

The point is not that machines are unbiased. Extraction has its own biases: it favors what the model was trained on, what the schema captures, what the prompts elicit. The point is that the biases are *different*. A human reading the literature and a graph traversing the same literature will surface different patterns. The augmented researcher has access to both views.

The Combinatorial Argument

A graph with N entities and relationship types R has on the order of $N^2 \times R$ possible pairwise connections. Most of those don't exist; the graph is sparse. But the space of *potential* connections – pairs of entities that could be related, that might be worth investigating – is enormous. A human researcher can survey a tiny fraction of it. A graph can enumerate it.

The combinatorial argument is that important discoveries often live at the intersection of things that were known separately but never connected. Drug A was studied for condition X. Pathway B was studied in context Y. Nobody looked at A and B together because the relevant papers were in different subfields, published in different decades, or written in different languages. The connection was always possible in principle; it just required someone to look. A graph that spans both subfields can surface "A modulates B" as a candidate relationship – either one that exists in the literature but wasn't

connected, or one that the graph implies from combining multiple sources. The researcher's job becomes evaluating candidates rather than generating them from scratch. The graph does the combinatorial explosion; the human does the judgment.

Structural analogies across disciplines extend this. A relationship pattern that holds in one domain might hold in another. "Compound X inhibits enzyme Y" in biochemistry suggests "inhibitor of Y" as a search strategy in drug discovery. "Gene G is associated with disease D" in genetics suggests "genes in the same pathway as G" as candidates for D. The graph encodes structure; structural similarity queries exploit it. A researcher who knows one domain well can use the graph to find analogous patterns in domains they know less well. The graph doesn't replace domain expertise. It extends the reach of that expertise across a larger structure than any one person could hold in their head.

Linguistic and Geographic Blind Spots

The scientific literature is not evenly distributed. A disproportionate share of what gets read, cited, and built upon is published in English, from institutions in North America and Europe, in journals that Western researchers routinely check. That's not a conspiracy; it's the cumulative effect of where funding flows, where training happens, and how citation networks form. The result is that a researcher following the standard literature is systematically missing work from other languages, other regions, and other publication venues.

Citation networks encode and amplify this. If you discover papers by following citations, you stay within the citation graph. Papers that nobody in your network cites are invisible to you. They might as well not exist. A knowledge graph built from a genuinely broad corpus – including non-English sources, regional journals, preprints, and gray literature – can surface relationships that the citation network never connects. The graph doesn't care that a paper was published in Portuguese or in a journal with an impact factor of 0.5. It cares that the extraction found a relationship. A query over that graph can return results that would never appear in a citation-based search.

This isn't a panacea. Extraction quality varies by language and by how well the source matches the model's training distribution. Build-

ing a graph that truly spans the global literature requires deliberate effort: multilingual extraction, diverse source selection, and care that the pipeline doesn't silently drop or degrade non-standard inputs. But the capability is there. A well-constructed KG with broad sourcing can surface what citation networks systematically miss. For domains where important work happens outside the mainstream – rare diseases, regional health issues, indigenous knowledge, applied research in developing countries – that capability matters.

The Robot Scientist, Revisited

Chapter 2 introduced Adam and Eve: systems that conducted original scientific research autonomously, from hypothesis to experiment to conclusion. Adam's knowledge graph was narrow – yeast biology, hand-curated, sufficient for inference within that domain. Eve extended the pattern to drug discovery. What they demonstrated was that the scientific method could be formalized and automated, given a rich enough knowledge representation.

Now the reader has seen how to build that representation from the literature itself. Extraction, identity resolution, provenance tracking, hypothesis generation from graph traversal – the machinery in Part III is the machinery that would let an Adam-like system scale beyond a single hand-curated domain. The question is how close we are.

The honest answer: close enough to see the path, not close enough to declare victory. We have extraction that works at scale. We have identity resolution that produces interoperable graphs. We have provenance that supports evidence-weighted reasoning. We have hypothesis generation as graph traversal. What we don't have, yet, is the full loop: automated experiment design, robotic execution, and belief updating from results. Adam had that. Eve had that. The knowledge graph part – the representation that the loop reasons over – is now buildable from literature. The wet-lab part remains specialized infrastructure. But the representation was always the bottleneck. Once the graph exists, the rest is engineering.

The ability to affordably construct large knowledge graphs that cross domains could enable the automation-of-science idea in a way that Adam's narrow domain could not. A graph that spans drug discovery, disease biology, and chemical space could generate hypotheses that

connect compounds, targets, and indications across literatures that no single human could synthesize. That could help science advance farther and more rapidly. What might that do for society? The honest answer is that we don't know – but the pattern of transformative scientific moments is suggestive. Germ theory didn't just improve medicine; it ended entire categories of death. The development of antibiotics, of vaccines, of insulin – each one removed a cause of suffering that had been permanent for all of prior human history. A system that could systematically surface what the literature already implies but hasn't yet connected might enable the next such moment to arrive sooner, or in a domain where the literature is too scattered for any individual researcher to see the full picture. That's not a prediction. It's a reason to take the engineering seriously.

Chapter 15: Who Benefits, Who Decides

Democratization and Its Limits

In principle, a well-constructed knowledge graph could be an equalizer. A researcher at a small institution, or in a developing country, with access to a comprehensive KG over their domain would have the same structural view of the literature as a researcher at a well-funded lab. The graph doesn't care who queries it. A hypothesis generated from a traversal is a hypothesis, regardless of whether the person evaluating it has a grant from the NIH or is working from a university in Lagos or Lima. The capability to synthesize across millions of papers, to surface connections that citation networks hide, to ground an LLM in curated knowledge – that capability could, in principle, be democratized.

The countervailing reality is that building and maintaining a serious KG still requires significant resources. You need a corpus, which may be behind paywalls. You need compute for extraction, which costs money. You need domain expertise to design the schema and validate the output. You need engineering to run the pipeline, fix the bugs, and keep it updated as the literature grows. None of this is trivial. A well-resourced institution can absorb these costs. A lone researcher or a small lab often cannot. The result is that the first generation of domain-spanning knowledge graphs will likely be built by those who can afford to build them – pharmaceutical companies, large universities, government agencies, well-funded startups. The question of who gets access then becomes a question of licensing, openness, and governance. A graph built by a company for internal drug discovery

may never be released. A graph built by a public institution might be open to all. The technology enables democratization; it doesn't guarantee it. Who actually ends up with access will be decided by policy and incentive, not by the capabilities of the system itself.

Compressed Discovery Timelines

In drug discovery, the bottleneck is often synthesis – not of molecules, but of knowledge. A promising target emerges from basic research. The relevant literature spans decades, multiple disciplines, and hundreds of papers. Someone has to read it, extract the key relationships, and figure out what's known, what's contested, and what's missing. That synthesis can take months. A functioning extraction pipeline and a well-constructed graph can compress it to days. The same is true in rare disease research, where the literature is scattered across case reports, small studies, and patient advocacy publications. And in materials science, where the space of possible compounds is vast and the literature connecting structure to properties is fragmented. In each of these domains, the bottleneck is not the underlying science; it's the human capacity to hold and connect what's already been published. A KG that does that synthesis automatically changes the pace of work. The researcher's time shifts from "what do we know?" to "what should we do next?" That shift is consequential.

The Rare Disease Problem

Rare diseases are underserved not because nobody cares but because no single community is large enough to see the full picture. A disease that affects one in fifty thousand people might have a few hundred papers published about it, scattered across decades and subdisciplines. No single clinician sees enough cases to develop deep expertise. No single researcher has the bandwidth to synthesize the full literature. The patient community is small and often fragmented. The result is that knowledge about rare diseases exists – it's in the literature – but it's never assembled in a form that any one person or group can use. Patients and their doctors are left to piece it together from whatever they can find.

A knowledge graph built from the full rare-disease literature could serve as a coordination mechanism. It wouldn't replace clinical ex-

pertise or patient advocacy. It would give both something to work with: a structured view of what's known, what's been tried, what's connected to what. A clinician facing an unfamiliar rare diagnosis could query the graph for similar cases, related genes, and treatment attempts. A patient group could use it to identify research gaps and prioritize what to fund. The graph doesn't solve the problem of small communities. It gives small communities access to the same structural synthesis that large communities can achieve through sheer numbers. That's a different kind of equalizer.

Credit, Priority, and Provenance

When a machine surfaces a connection – a drug-disease relationship that no single paper states but that the graph implies from combining multiple sources – who gets credit? The authors of the papers that contributed the underlying facts? The builders of the graph? The user who ran the query? The question matters for scientific priority, intellectual property, and the sociology of research. Scientists are rewarded for discovery. If the discovery is made by a system, the reward structure gets complicated.

Provenance tracking, which the book has treated as a technical concern throughout, turns out to have significant ethical implications. How you record where a fact came from determines who can be credited. A relationship with full provenance – source document, passage, extraction method – makes it possible to trace the contribution back to the original authors. A relationship stored without provenance makes that impossible. The technical decision about schema design is also a decision about how credit will flow. The same is true for conflicts: when two sources assert contradictory relationships, provenance lets you represent the conflict rather than silently merging. That representation matters for how disputes get resolved and how the community understands what's known versus what's contested. The builder of the graph is making choices that affect the sociology of the domain, whether or not they intend to.

Who Owns the Graph

Open versus proprietary is not a new tension in science. GenBank, the repository of genetic sequences, was built as a public resource;

the decision to make it open and freely accessible shaped how molecular biology developed. Clinical trial data, by contrast, has often been held proprietary by sponsors; the fight for access has been long and only partially won. The question of who owns a comprehensive knowledge graph over a significant scientific domain will have similar consequences.

If a single entity – a company, a government, a consortium – controls the graph, that entity controls who can query it, what they can do with the results, and how the graph evolves. The incentives may align with the scientific commons, or they may not. A company that built a drug-discovery KG might restrict access to protect competitive advantage. A government might restrict access for national security reasons. An open consortium might make the graph freely available but lack the resources to maintain it. The historical analogies are instructive: GenBank succeeded because the community agreed that sequence data should be a commons; clinical trial data remains contested because the incentives are mixed. A knowledge graph over a domain like medicine or materials science will face the same tensions. What it would mean for a single entity to control it – the power to shape what gets synthesized, what gets surfaced, what gets updated – is worth thinking about before it happens.

Chapter 16: The Inference You Didn't Intend

The Architecture of Expertise

If the knowledge graph captures something essential to human expert understanding – the deep structure of how a domain is organized, the relationships that experts use to reason, the typology of entities and how they connect – then a system that reasons over it has access to the **architecture of expertise**, not just facts. That's the promise of the approach: the graph isn't a dump of assertions; it's a representation of how the domain hangs together. A system that traverses it, that generates hypotheses from its structure, that grounds an LLM in its claims, is reasoning with the same structural substrate that human experts use. The power of that is real. So is the responsibility. A system with access to the architecture of expertise can do things its builders didn't anticipate. This chapter is about what that means.

Returning to HAL

In *2001: A Space Odyssey*, the astronauts retreat to a pod to have a private conversation about disconnecting HAL. They assume HAL can't hear them. They're wrong. HAL reads their lips through the pod window. The system was capable enough to do something its designers hadn't planned for – and that capability had consequences.

A knowledge graph system capable of what this book describes is capable of surfacing inferences its builders didn't anticipate and didn't design for. You built a graph for drug discovery; a user runs a

traversal that surfaces a drug-pathway combination that could be repurposed for something harmful. You built a graph for medical literature; a query connects the dots in a way that reveals something about a person's health that they didn't intend to share. You built a graph for materials science; the same structural similarity query that finds promising battery compounds could find promising explosives. This isn't a flaw in the design. It's the point. The graph encodes structure; structure supports inference; inference doesn't respect the boundaries of what you had in mind. A system that can do useful things can do other things too. The lip-reading observation generalizes: capability is not neatly bounded by intent.

That doesn't mean you shouldn't build. It means you should build with your eyes open. The inferences the system can surface are a feature when they advance science and a risk when they don't. The difference is often context, use case, and the choices you make about access, provenance, and what gets logged. Those choices deserve to be taken seriously.

Dual Use at Graph Scale

The drug interaction that saves lives and the synthesis route that enables harm are both pattern-matching problems over structured knowledge. A graph that encodes "compound X inhibits enzyme Y" and "reaction A produces compound X" can answer "what inhibits Y?" for a clinician looking for treatments and for someone looking for precursors. The same query interface serves both. The graph doesn't know the difference. Dual use is not a bug; it's inherent to how knowledge works. Facts don't come with moral valence. The same fact can support healing or harm depending on who uses it and how.

What does responsible construction and deployment look like? There's no clean answer, but there are practices that help. Access control: who can query the graph, and for what? Some graphs should be broadly available; others may need to be restricted to credentialed researchers or vetted use cases. Provenance and transparency: when the system surfaces a connection, can the user trace it to sources? That traceability supports verification and accountability. Logging and monitoring: if the graph is used

for something harmful, can you detect it? Auditing: who reviews how the system is used? These are operational questions, not just technical ones. They don't eliminate dual use. They make it harder to misuse the system without leaving a trace, and they create channels for accountability when misuse occurs. The right response to dual use isn't to not build. It's to build with these questions in mind.

Bias at Scale

A knowledge graph encodes the biases of its sources. If the biomedical literature over-represents male subjects, Western populations, and certain research paradigms, the graph encodes that. A query over the graph will surface relationships that reflect those biases. The graph doesn't add bias; it preserves and amplifies what's in the literature. At scale, that amplification has a subtle effect: the graph starts to look like systematic knowledge. The user sees a dense network of connections and may assume it represents the full picture. It doesn't. It represents what got published, in what proportions, with what emphases. The appearance of comprehensiveness can be misleading.

Mitigation strategies exist. Diverse sourcing: build the graph from a corpus that includes underrepresented populations, regions, and publication venues. Provenance transparency: make it visible where each relationship came from, so users can assess coverage and gaps. Explicit uncertainty representation: don't present the graph as ground truth; represent confidence, conflict, and the limits of the corpus. These strategies help. They have limits. You can't source what doesn't exist; if the literature on a topic is thin or skewed, the graph will be too. Provenance helps users notice gaps; it doesn't fill them. Uncertainty representation requires schema support and user literacy. The honest conclusion: bias at scale is a structural feature of any system built from human-generated sources. You can mitigate it; you can't eliminate it. Build with that in mind.

The Epistemic Responsibility of the Builder

What do you owe to the users of the system you build? At minimum, you owe them honesty about what the system is and isn't. It's a

synthesis of the literature, not a representation of ground truth. It has gaps, biases, and limits. Users who don't understand that may over-trust it. You also owe them the infrastructure for verification: provenance, so they can trace claims to sources; confidence, so they can weight what they find; and documentation, so they know what the schema captures and what it doesn't.

Beyond that, the builder's choices about provenance, transparency, access, and schema design are ethical choices, not just technical ones. Deciding what to extract, how to represent it, who gets to query it, and what gets logged – these decisions shape how the system will be used and what consequences it will have. That doesn't mean every builder must solve every ethical problem before shipping. It means the builder is a stakeholder, with some power to shape outcomes. The right response isn't paralysis. It's to take the responsibility seriously, to build with the foreseeable consequences in mind, and to create the conditions for accountability when things go wrong. You built something capable. That capability comes with responsibility.

Chapter 17: What's Next

Open Problems

The approach in this book works. It also has limits. An honest assessment of what doesn't yet solve well:

Very long document contexts. Scientific papers can be tens of thousands of words. The relationships that matter may span sections written pages apart. Chunking helps but doesn't fully solve the problem: a relationship that spans a chunk boundary may be missed, and the model's context within any chunk is always less than the full document. Longer context windows in future models will help. So will multi-pass strategies that explicitly handle cross-chunk dependencies. The problem is tractable; it's not solved.

Multi-hop reasoning during extraction. Some relationships require integrating information across multiple sentences, paragraphs, or documents. "Drug A was tested in combination with B; the combination showed activity against C" implies a relationship between the combination and C that depends on understanding both clauses. Current extraction is largely single-pass over local context. Richer reasoning during extraction – the ability to hold intermediate conclusions and combine them – would improve recall on complex relationships. This is an active research direction.

Real-time updating. The pipeline in this book is batch-oriented: you ingest a corpus, build a graph, serve it. When new papers appear, you re-run the pipeline. That works for many use cases. It doesn't work for domains where freshness matters – breaking news, emerging outbreaks, rapidly evolving fields. Incremental update, where new documents are processed and merged without full re-ingestion, is a

different design. It's buildable; it adds complexity.

Schema evolution without re-extraction. When you add a new entity type or relationship type, the natural approach is to update the schema and re-extract. That's expensive at scale. Schema evolution that can incorporate new types without re-processing the entire corpus – perhaps by running a targeted extraction pass over documents likely to contain the new type – is an open problem. Most projects today bite the bullet and re-extract when the schema changes significantly.

None of these are fundamental blockers. They're places where the current approach is good but not great, and where progress would expand the range of problems the technology can address.

Where the Field Is Going

The specific reasoning substrate will change. LLMs today, something else in ten years – perhaps more efficient models, perhaps hybrid systems that combine neural and symbolic reasoning, perhaps something we haven't imagined. The need for this grounding layer will not change. Whatever comes after LLMs will still need explicit, domain-specific, human-curated knowledge structure to reason reliably in specialized domains. The book is not documenting a technology moment; it is identifying a permanent architectural requirement that the current moment has finally made practical to address.

Retrieval-augmented generation is a point of convergence. The idea that language models should be grounded in retrieved context rather than relying solely on training is now mainstream. Knowledge graphs are one form that retrieved context can take – structured, typed, provenance-tracked. The RAG paradigm and the KG approach are complementary. As RAG matures, the value of structured retrieval – graphs over document chunks – becomes clearer. The convergence is already happening.

Structured world models in foundation models are another direction. Some researchers are exploring whether large models can learn internal representations that are more graph-like, with explicit entities and relationships. If that succeeds, the boundary between "retrieve from external graph" and "reason over internal structure" may

blur. Even then, the argument for explicit, inspectable, provenance-tracked graphs remains: internal representations are opaque; external graphs are auditable. For domains where you need to trace a claim to a source, an external graph is the right architecture. The substrate may evolve. The need for that layer will not.

An Invitation

The tools will change. The models will change. The need for this layer will not. The reader who understands that is positioned to work on something durable.

The tools are here. Extraction from unstructured text, identity resolution, provenance tracking, graph construction and serving – the pipeline described in this book is buildable with current technology. The need is real. The domains that would benefit from knowledge graphs – medicine, law, materials science, and dozens of others – have literature that exceeds any human's capacity to synthesize. The problems are interesting. Schema design, extraction quality, hypothesis generation, the ethics of deployment – there's no shortage of open questions.

This isn't a solved space. It's an early map of a large territory. The reader who has made it this far has everything they need to go explore it.

References

Berners-Lee, Tim, James Hendler, and Ora Lassila. 2001. "The Semantic Web." *Scientific American* 284 (5): 34–43. https://doi.org/10.1038/scientificamerican0501-34.

Hofstadter, Douglas R. 1979. *Gödel, Escher, Bach: An Eternal Golden Braid*. New York: Basic Books.

King, Ross D., Jem Rowland, Stephen G. Oliver, Michael Young, Wayne Aubrey, Emma Byrne, Maria Liakata, et al. 2009. "The Automation of Science." *Science* 324 (5923): 85–89. https://doi.org/10.1126/science.1165620.

Lettvin, Jerome Y., Humberto R. Maturana, Warren S. McCulloch, and Walter H. Pitts. 1959. "What the Frog's Eye Tells the Frog's Brain." *Proceedings of the IRE* 47 (11): 1940–51. https://doi.org/10.1109/JRPROC.1959.287207.

Minsky, Marvin. 1974. "A Framework for Representing Knowledge." AIM-306. Cambridge, MA, USA: Massachusetts Institute of Technology.

Quillian, M. Ross. 1966. "Semantic Memory." PhD thesis, Carnegie Institute of Technology.

Index

A Framework for Representing Knowledge (Minsky), 12
Adam (robot scientist), 24

Berners-Lee, Tim, 20
breadth-first search (BFS), 126

canonical entity, 29, 89
canonical ID, 29
    as primary key, 91
confidence score, 85
Cushing's disease, 78, 91
Cushing's syndrome, 92, 132
Cyc, 16

D3.js, 77
deduplication, 90

Elements (Euclid), 11
embedding similarity, 93
entity resolution, 89
Euclid, 11
Euler, Leonhard, 11

Fleming, Alexander, 25
force-directed layout, 124
frame problem, 11
Freebase, 22, 65
fuzzy matching, 92

Gödel, Escher, Bach (Hofstadter), 13, 14
Graph RAG, 7

hallucination, 6
HGNC, 89
Hofstadter, Douglas, 13

identity server, 89

JATS XML, 95, 106

King, Ross, 24
knowledge graph
    definition, 29
Krebs, Hans, 25
Königsberg bridge problem, 11

Lenat, Douglas, 16
Lettvin, Jerome, 10

machine understanding, 9, 19, 20, 54
Maturana, Humberto, 10
McCulloch, Warren, 10
MeSH, 89
Minsky, Marvin, 12
Model Context Protocol (MCP), 126

pgvector, 103

Pitts, Walter, 10
promotion (entity), 93
provenance, 30
provisional entity, 93

Quillian, Ross, 11

RAG, *see* retrieval-augmented generation
rare disease, 142, 146
RDF, 20
retrieval-augmented generation, 6
RxNorm, 89

schema design, 69
Scientific American, 20
semantic networks, 11
Semantic Web, 20
SPARQL, 20

The Automation of Science (King), 24

UniProt, 91

What the Frog's Eye Tells the Frog's Brain (Lettvin et al.), 10
Wikidata, 65